



INSTITUTO POLITECNICO NACIONAL “IPN”



ESCUELA SUPERIOR DE COMPUTO “ESCOM”

LANDEROS CORTES PEDRO JONAS

SISTEMAS DISTRIBUIDOS

TAREA 9

2023630402

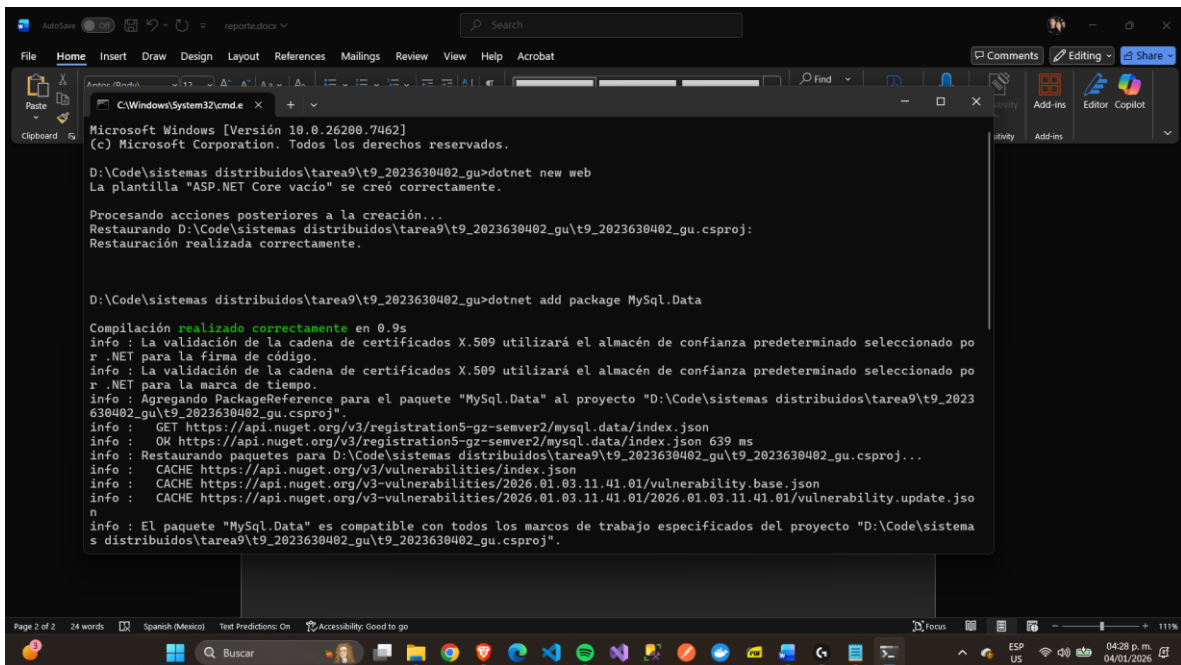
Indice

Creacion de microservicios.....	4
Creacion de solución de microservicio para usuarios	4
Creacion de base de datos para usuarios	4
Creacion de la base de datos artículos	6
Creacion de solucion	7
Creacion de base de datos microservicio gestión de compras	8
Creacion de solución	9
Creacion de solución para el servicio web	10
Pruebas locales del flujo básico	10
Instalacion/Comprobacion herramientas CI/CD.....	13
Creacion en azure	14
Resource Group.....	14
ACR	14
Claves de acceso.....	15
AKS.....	15
Conexión de kubernetes.....	16
MySQL PaaS	17
Creacion mysql-secret	18
Creacion de secrets	18
Deployment de contenedores docker al ACR.....	19
GU	19
GA	20
GC.....	21
SW.....	22
Subida de archivos del front	22
Gateway.....	23
Subida de certificado auto firmado y creación de balanceador de carga que expone una ip pública.....	23
Pruebas front desde ip publica	24

Pruebas backend	43
Registrar usuario.....	43
Login.....	43
Obtener usuario	44
Crear articulo	44
Consultar artículos	45
Comprar articulo	45
Consulta carrito.....	46
Modificar carrito	46
Eliminar articulo del carrito	47
Eliminar carrito	47
Explicacion de microservicios	47
Requerimientos funcionales del Front-End	51
Conclusión	54

Creacion de microservicios

Creacion de solución de microservicio para usuarios



```
Microsoft Windows [Versión 10.0.26200.7462]
(c) Microsoft Corporation. Todos los derechos reservados.

D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>dotnet new web
La plantilla "ASP.NET Core vacío" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu\t9_2023630402_gu.csproj:
Restauración realizada correctamente.

D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>dotnet add package MySql.Data

Compilación realizada correctamente en 0.9s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "MySql.Data" al proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu\t9_2023630402_gu.csproj".
info : GET https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json 639 ms
info : Restaurando paquetes para D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu\t9_2023630402_gu.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.01.03.11.41.01/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.01.03.11.41.01/vulnerability.update.json
info : El paquete "MySql.Data" es compatible con todos los marcos de trabajo especificados del proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu\t9_2023630402_gu.csproj".
```

Creacion de base de datos para usuarios

Script

CREATE DATABASE IF NOT EXISTS t9_2023630402_bdgu;

USE t9_2023630402_bdgu;

CREATE TABLE usuarios (

id_usuario INT AUTO_INCREMENT PRIMARY KEY,

password VARCHAR(64) NOT NULL, -- aquí guardas el hash

token VARCHAR(20),

email VARCHAR(100) NOT NULL,

nombre VARCHAR(100) NOT NULL,

apellido_paterno VARCHAR(100) NOT NULL,

apellido_materno VARCHAR(100),


```

fecha_nacimiento DATETIME NOT NULL,

telefono BIGINT,

genero CHAR(1),

UNIQUE KEY usuarios_email_ux (email)
) ENGINE=InnoDB;

CREATE TABLE fotos_usuarios (

id_foto INT AUTO_INCREMENT PRIMARY KEY,

foto LONGBLOB,

id_usuario INT NOT NULL,

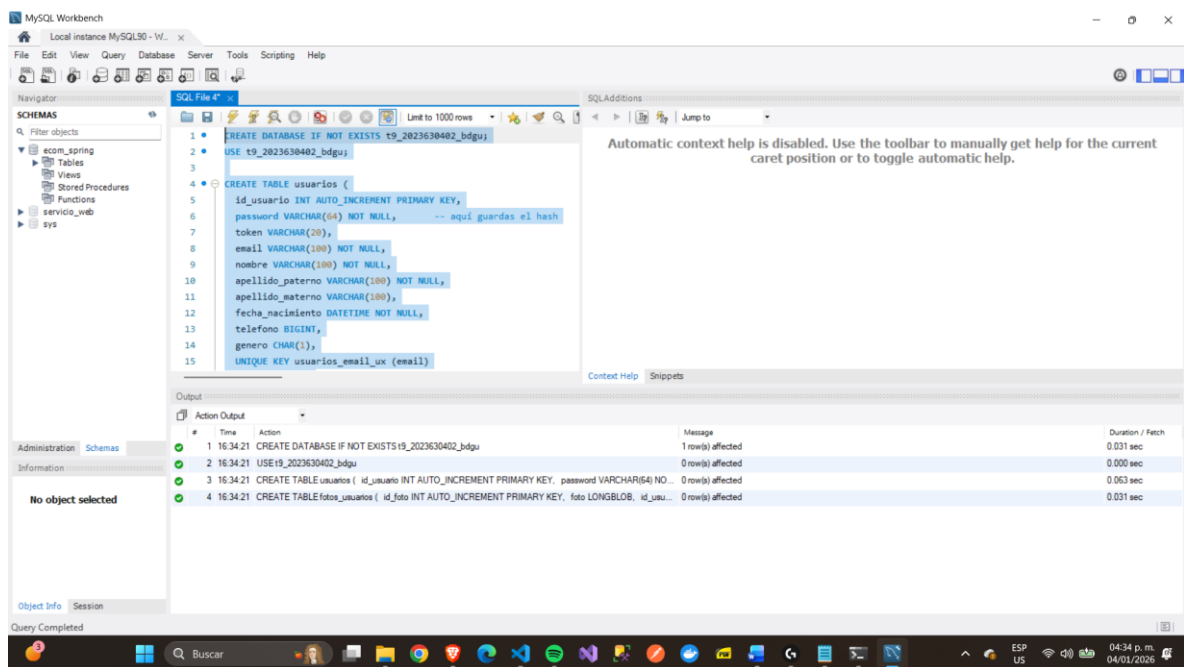
CONSTRAINT fk_fotos_usuarios_usuario

FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)

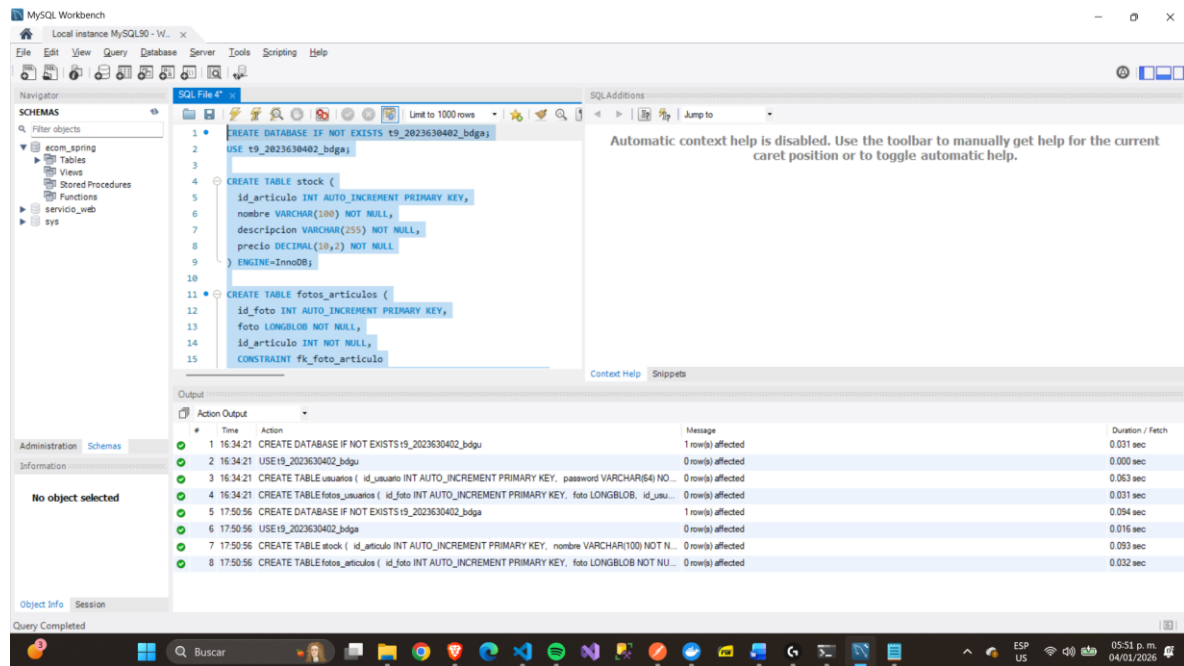
ON DELETE CASCADE

) ENGINE=InnoDB;

```



Creacion de la base de datos artículos



Script

```
CREATE DATABASE IF NOT EXISTS t9_2023630402_bdga;
```

```
USE t9_2023630402_bdga;
```

```
CREATE TABLE stock (
  id_articulo INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255) NOT NULL,
  precio DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;
```

```
CREATE TABLE fotos_articulos (
  id_foto INT AUTO_INCREMENT PRIMARY KEY,
  foto LONGBLOB NOT NULL,
  id_articulo INT NOT NULL,
```

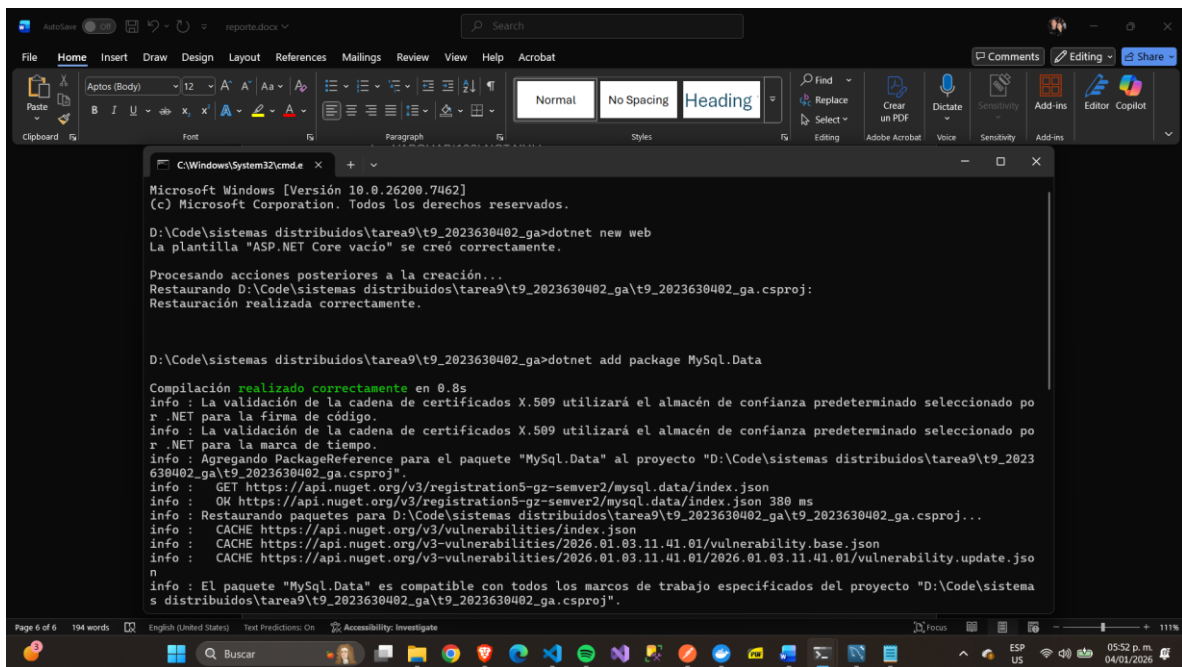
CONSTRAINT fk_foto_articulo

FOREIGN KEY (id_articulo) REFERENCES stock(id_articulo)

ON DELETE CASCADE

) ENGINE=InnoDB;

Creacion de solucion



```
Microsoft Windows [Versión 10.0.26280.7462]
(c) Microsoft Corporation. Todos los derechos reservados.

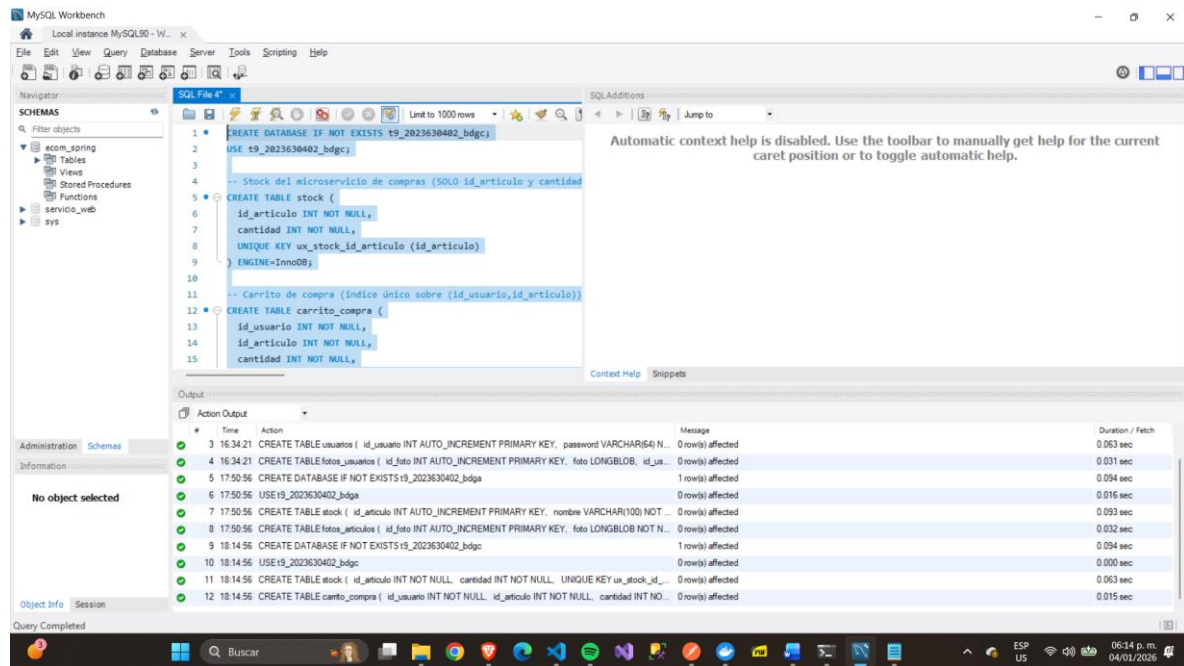
D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga>dotnet new web
La plantilla "ASP.NET Core vacío" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga\t9_2023630402_ga.csproj:
Restauración realizada correctamente.

D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga>dotnet add package MySQL.Data

Compilación realizado correctamente en 0.8s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "MySQL.Data" al proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga\t9_2023630402_ga.csproj".
info : GET https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json 380 ms
info : Restaurando paquetes para D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga\t9_2023630402_ga.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.01.03.11.41.01/vulnerability.base.json
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.01.03.11.41.01/2026.01.03.11.41.01/vulnerability.update.json
info : El paquete "MySQL.Data" es compatible con todos los marcos de trabajo especificados del proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_ga\t9_2023630402_ga.csproj".
```

Creacion de base de datos microservicio gestión de compras



Script

```
CREATE DATABASE IF NOT EXISTS t9_2023630402_bdgc;
```

```
USE t9_2023630402_bdgc;
```

```
-- Stock del microservicio de compras (SOLO id_articulo y cantidad)
```

```
CREATE TABLE stock (
```

```
  id_articulo INT NOT NULL,
```

```
  cantidad INT NOT NULL,
```

```
  UNIQUE KEY ux_stock_id_articulo (id_articulo)
```

```
) ENGINE=InnoDB;
```

```
-- Carrito de compra (índice único sobre (id_usuario,id_articulo))
```

```
CREATE TABLE carrito_compra (
```

```
  id_usuario INT NOT NULL,
```

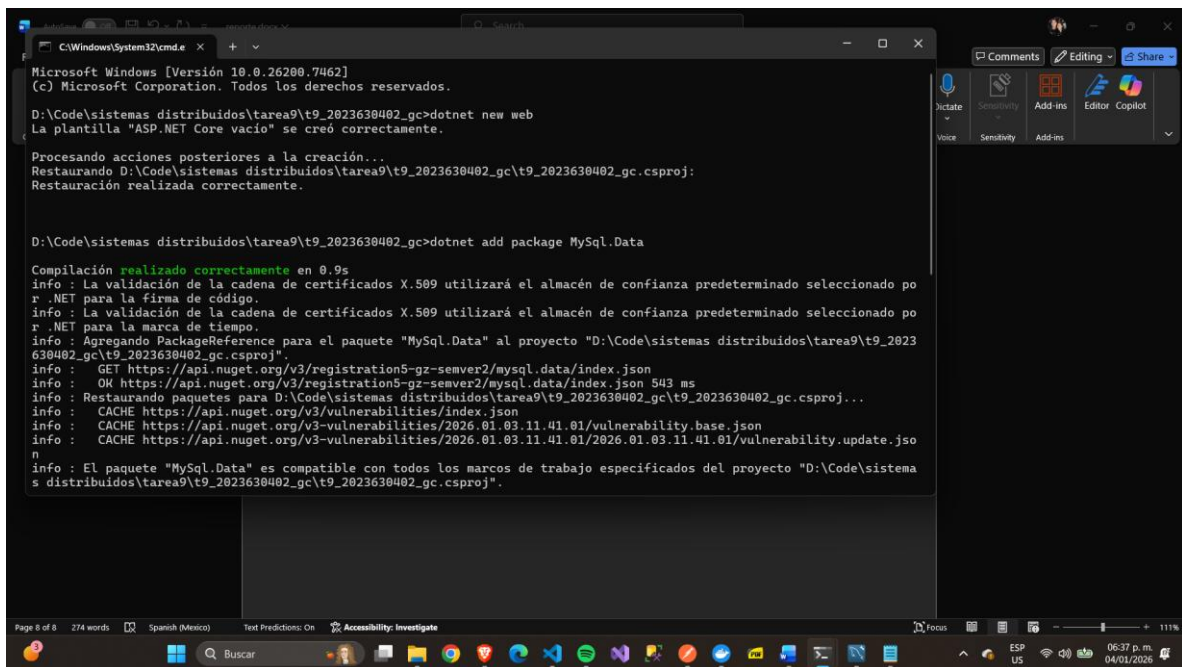
```
  id_articulo INT NOT NULL,
```

cantidad INT NOT NULL,

UNIQUE KEY ux_carrito_usuario_articulo (id_usuario, id_articulo)

) ENGINE=InnoDB;

Creacion de solución



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.26200.7462]
(c) Microsoft Corporation. Todos los derechos reservados.

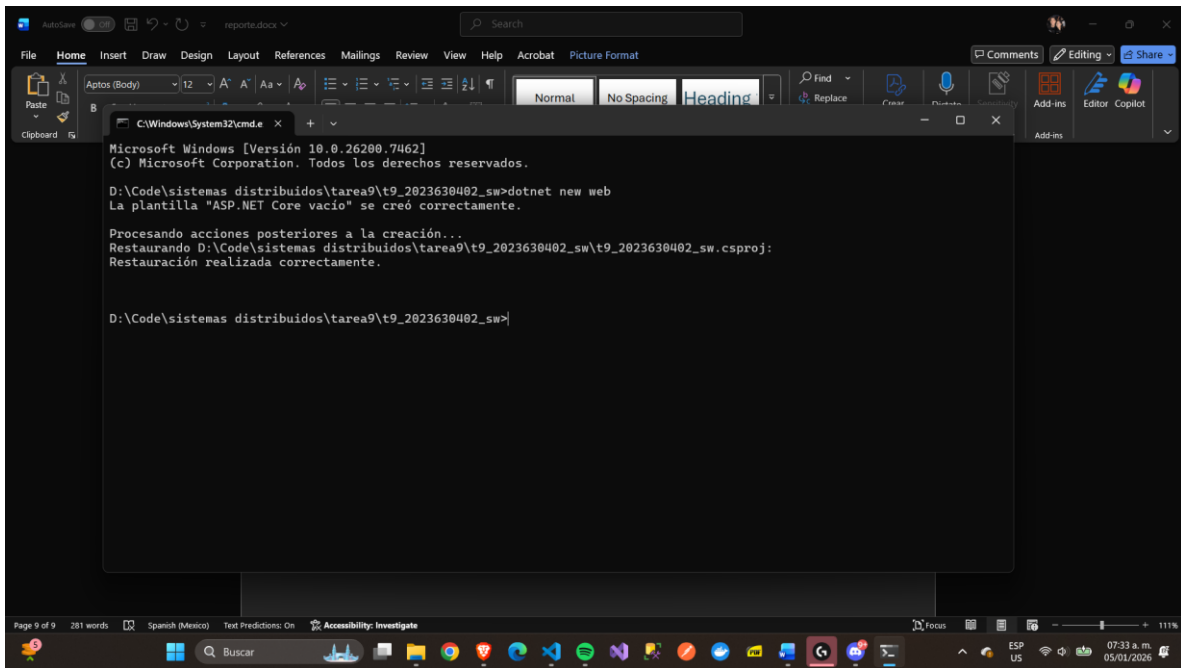
D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc>dotnet new web
La plantilla "ASP.NET Core vacío" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc\t9_2023630402_gc.csproj:
Restauración realizada correctamente.

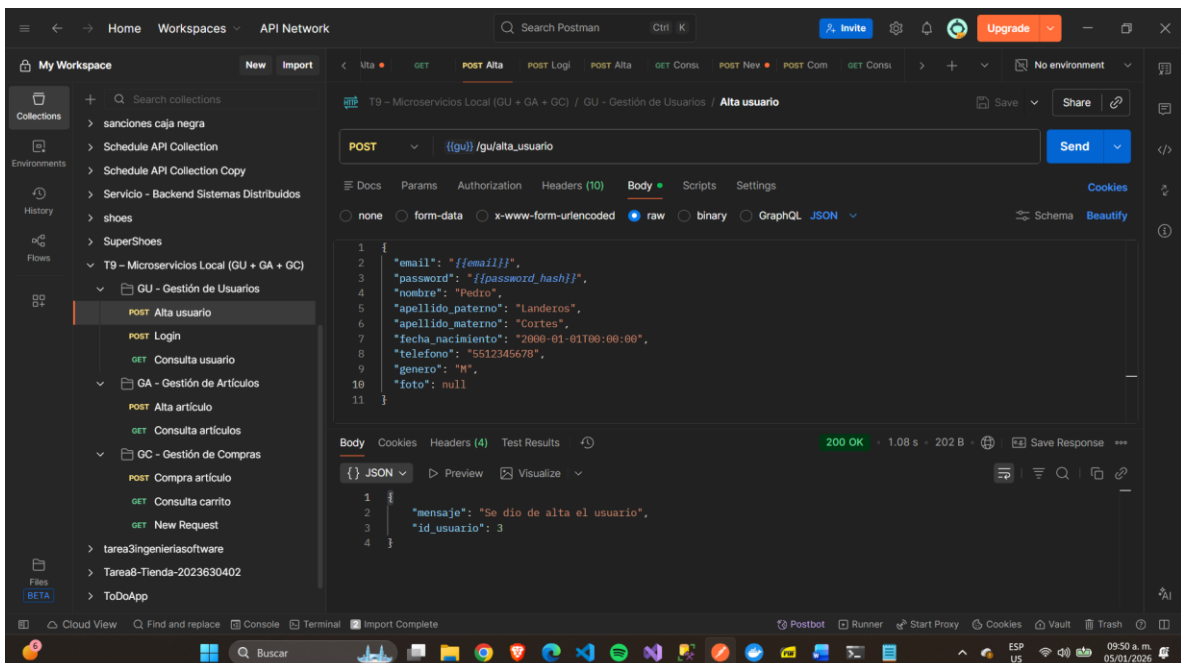
D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc>dotnet add package MySql.Data

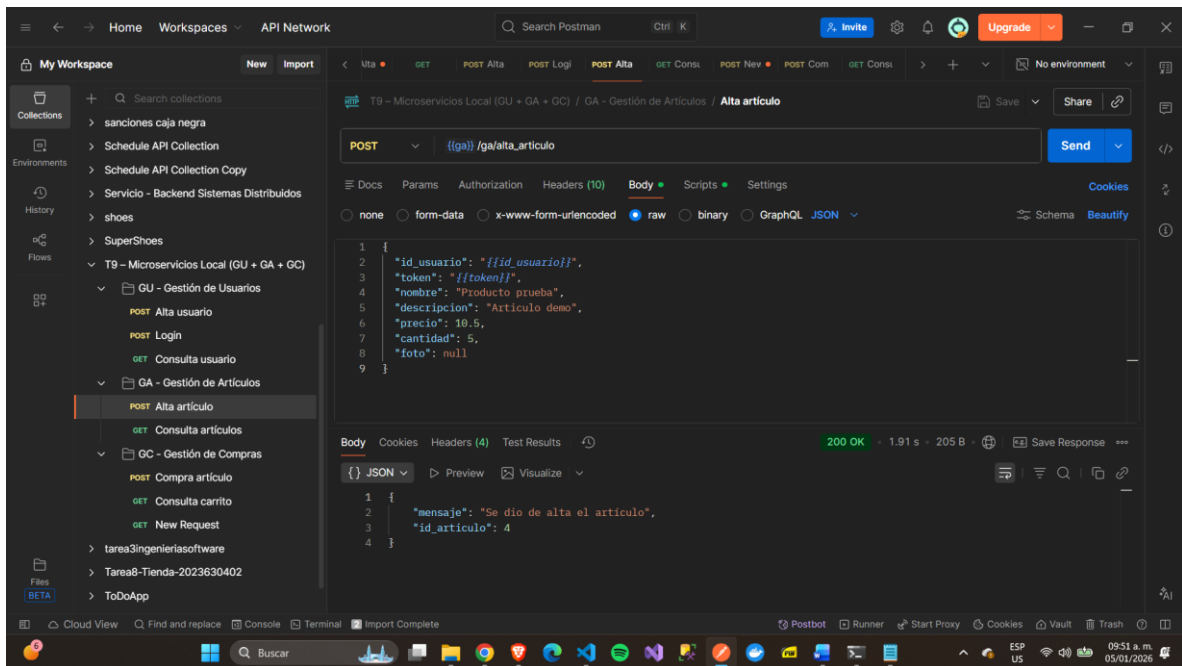
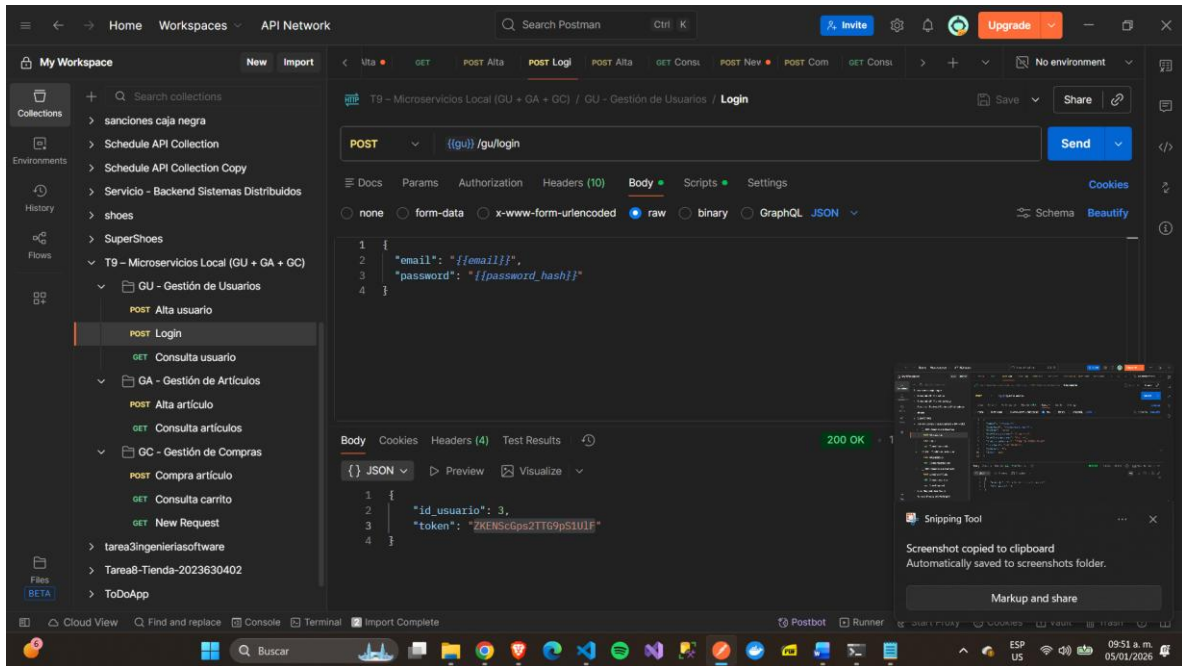
Compilación realizado correctamente en 0.9s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "MySql.Data" al proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc\t9_2023630402_gc.csproj".
info : GET https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/mysql.data/index.json 543 ms
info : Restaurando paquetes para D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc\t9_2023630402_gc.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.01.03.11.41.01/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.01.03.11.41.01/2026.01.03.11.41.01/vulnerability.update.json
info : El paquete "MySql.Data" es compatible con todos los marcos de trabajo especificados del proyecto "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gc\t9_2023630402_gc.csproj".
```

Creacion de solución para el servicio web



Pruebas locales del flujo básico





Postman interface showing a GET request to `/ga/consulta_articulos?palabra=Producto&id_usuario={{id_usuario}}&token={{token}}`. The request is successful (200 OK) and returns a JSON response:

```
{  "id_articulo": 4,  "nombre": "Producto prueba",  "descripcion": "Articulo demo",  "precio": 10.50,  "foto": null}
```

The interface also shows the Params tab with the following parameters:

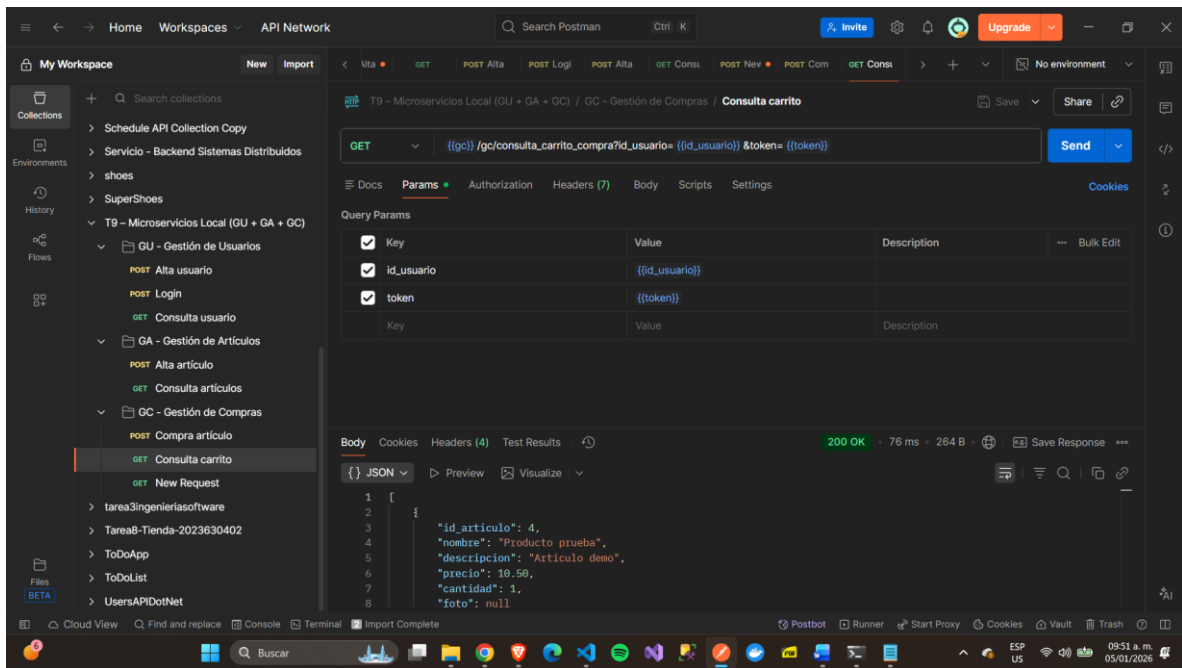
Key	Value	Description
palabra	Producto	
id_usuario	{{id_usuario}}	
token	{{token}}	

Postman interface showing a POST request to `/gc/compra_articulo`. The request is successful (200 OK) and returns a JSON response:

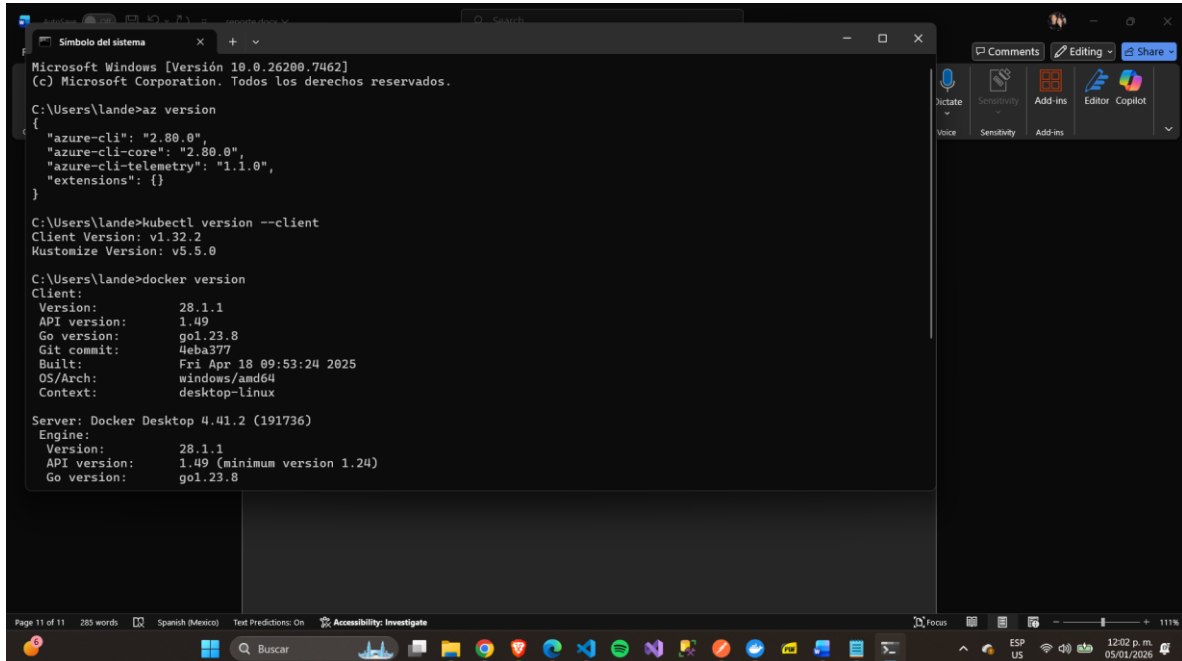
```
{  "mensaje": "Articulo agregado al carrito"}
```

The interface also shows the Body tab with the following JSON payload:

```
{  "id_usuario": "{{id_usuario}}",  "token": "{{token}}",  "id_articulo": "{{id_articulo}}",  "cantidad": 1}
```

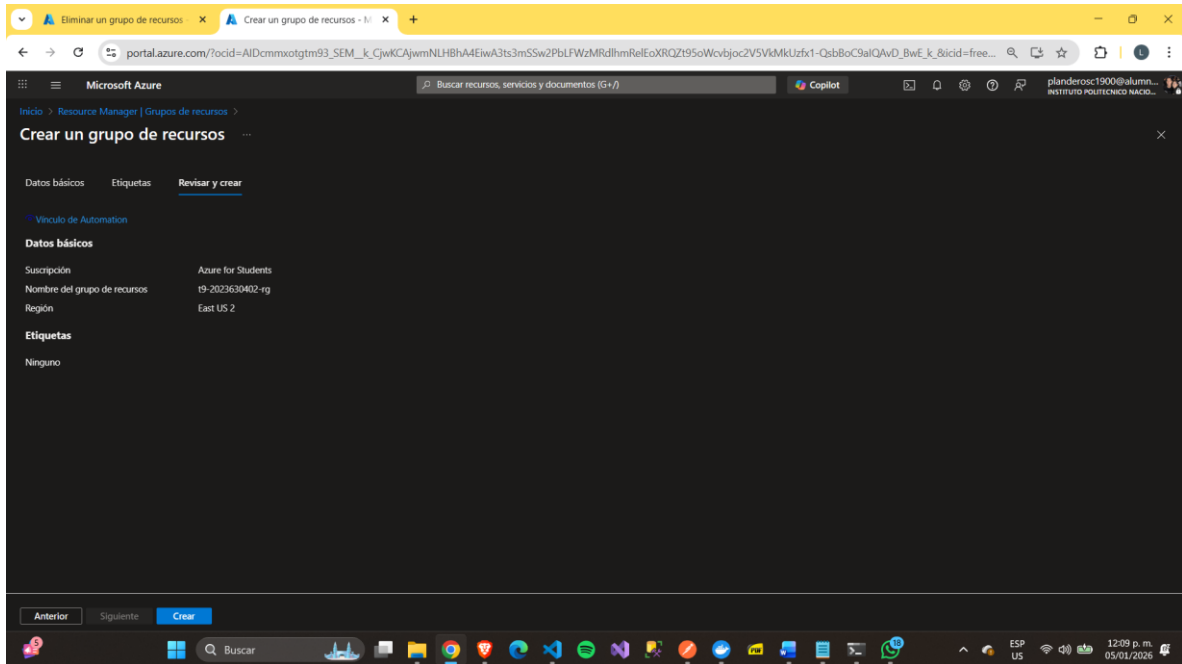



Instalacion/Comprobacion herramientas CI/CD

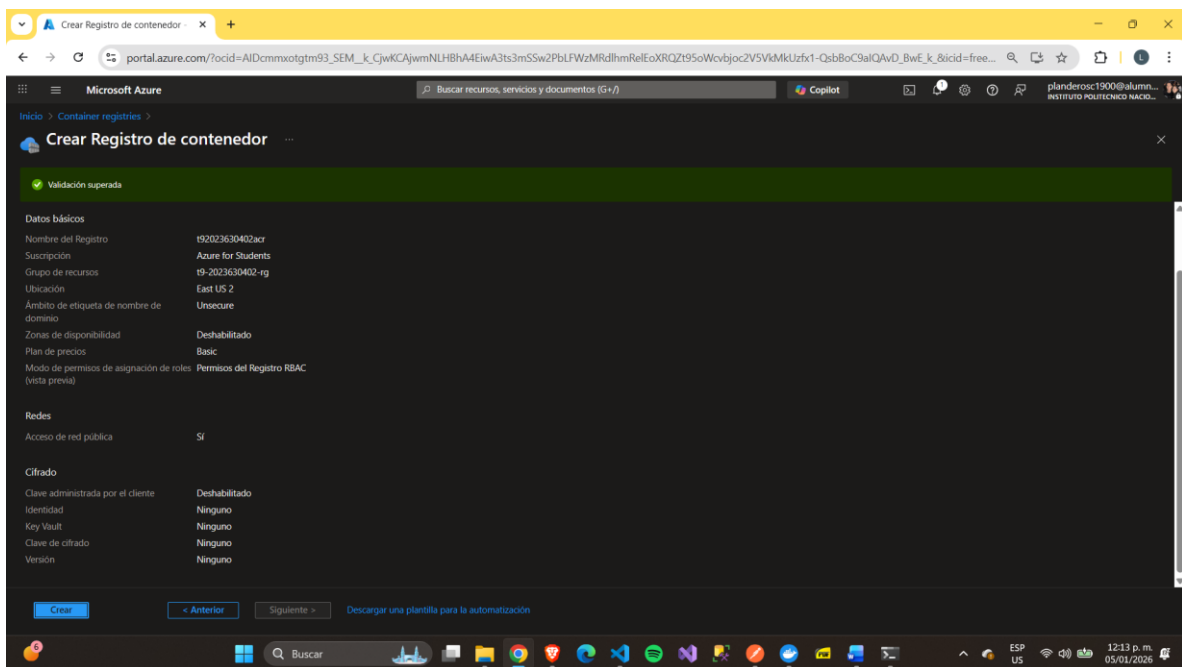


Creacion en azure

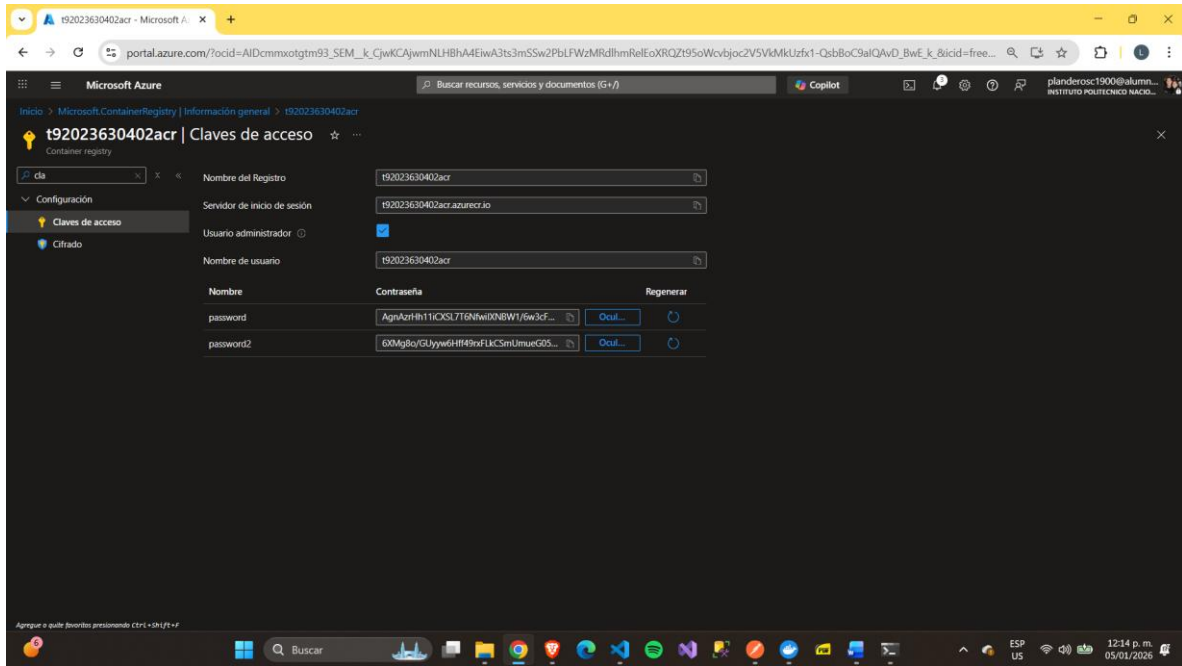
Resource Group



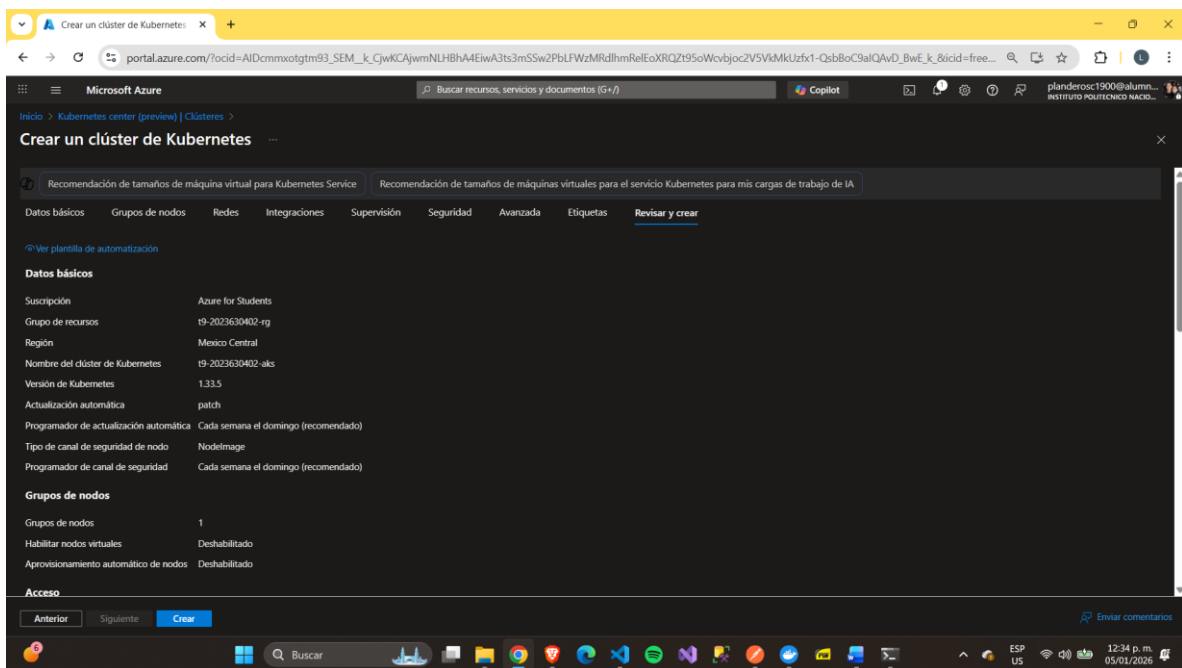
ACR

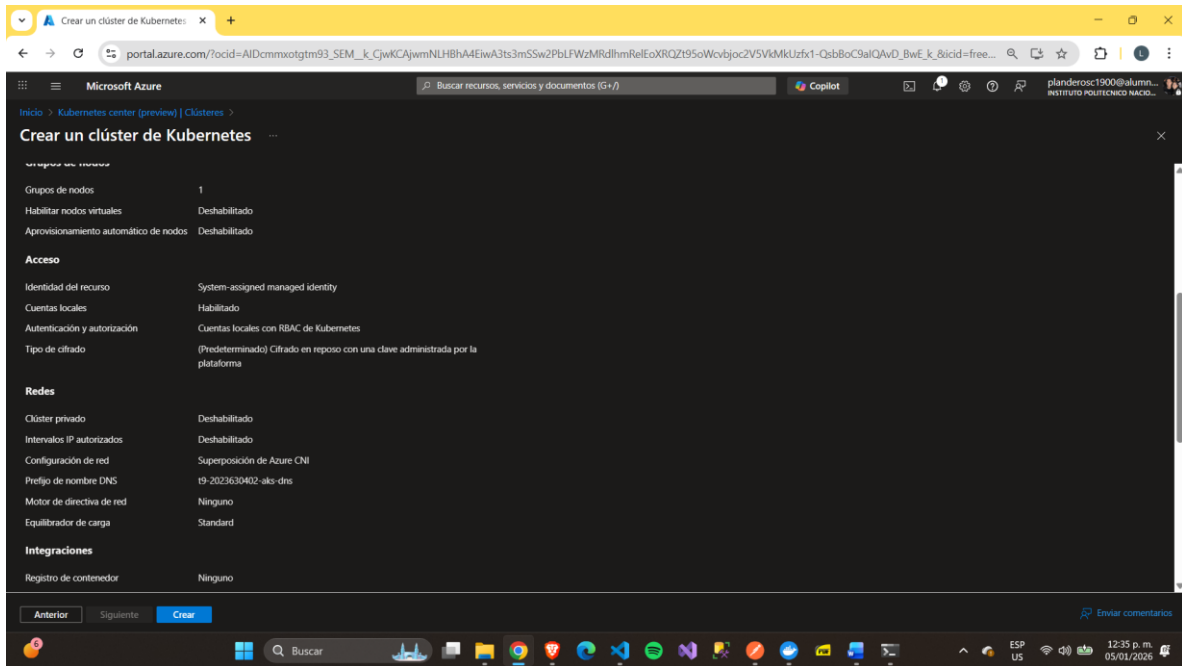


Claves de acceso

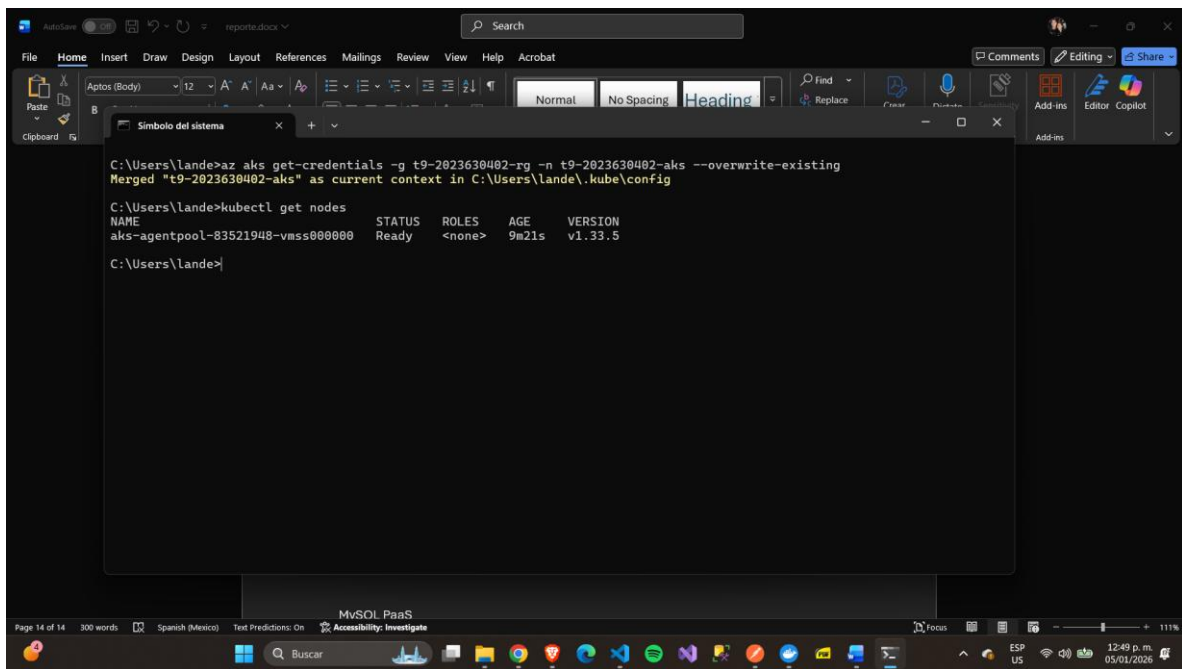


AKS

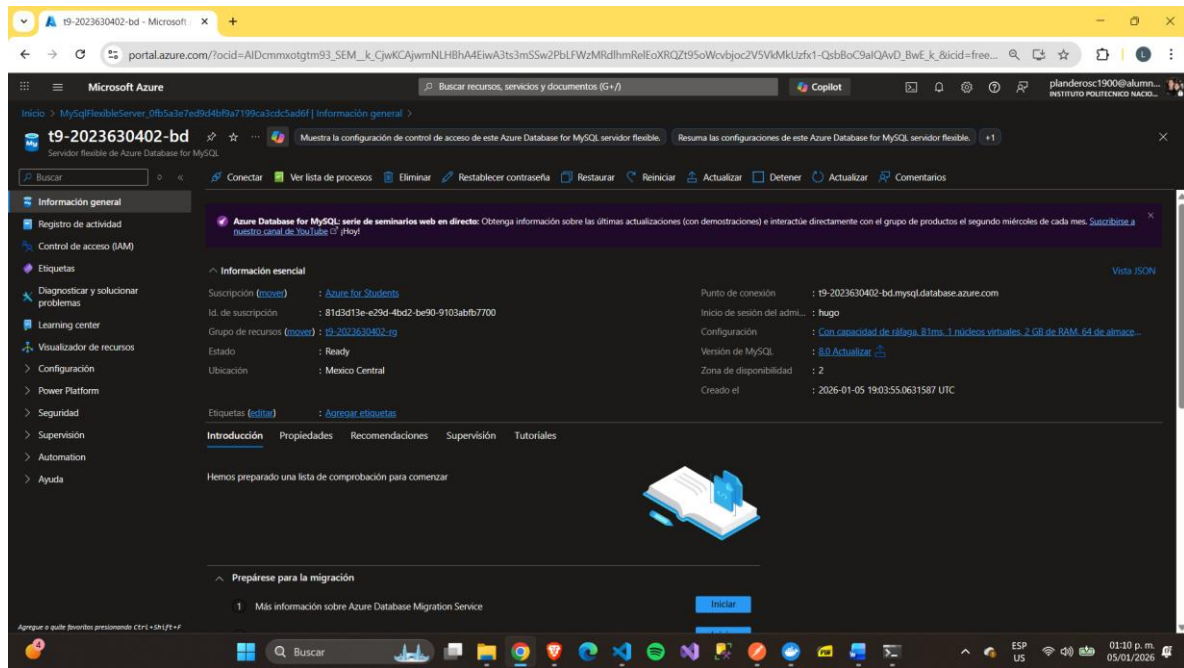




Conexión de kubernetes

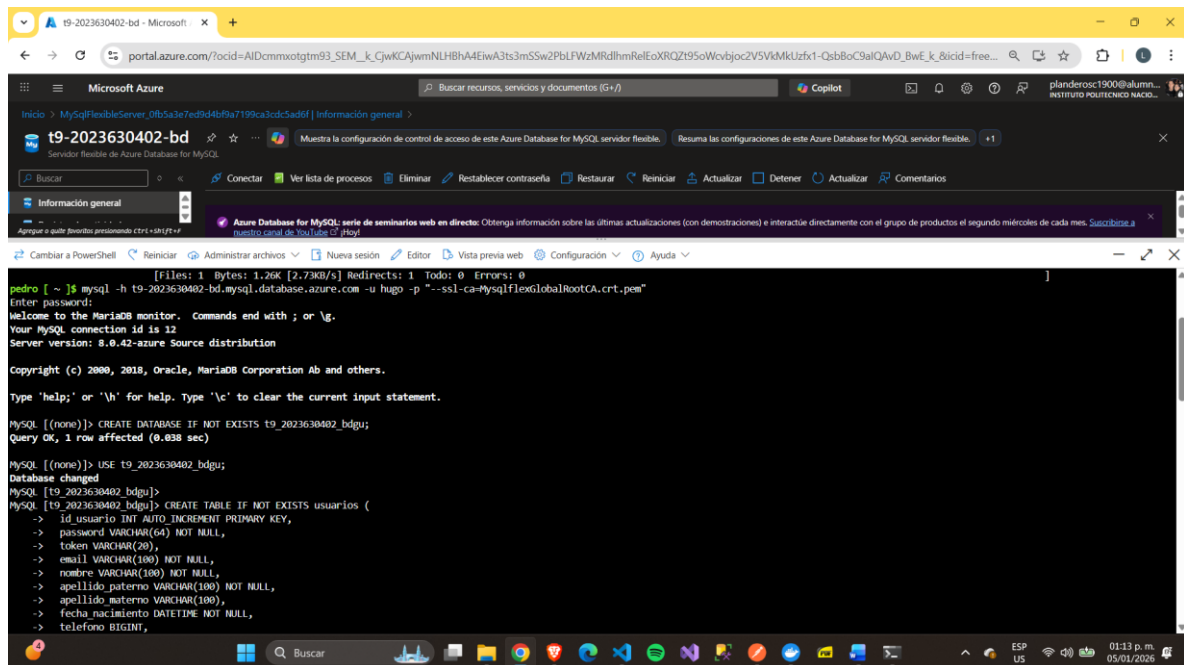


MySQL PaaS

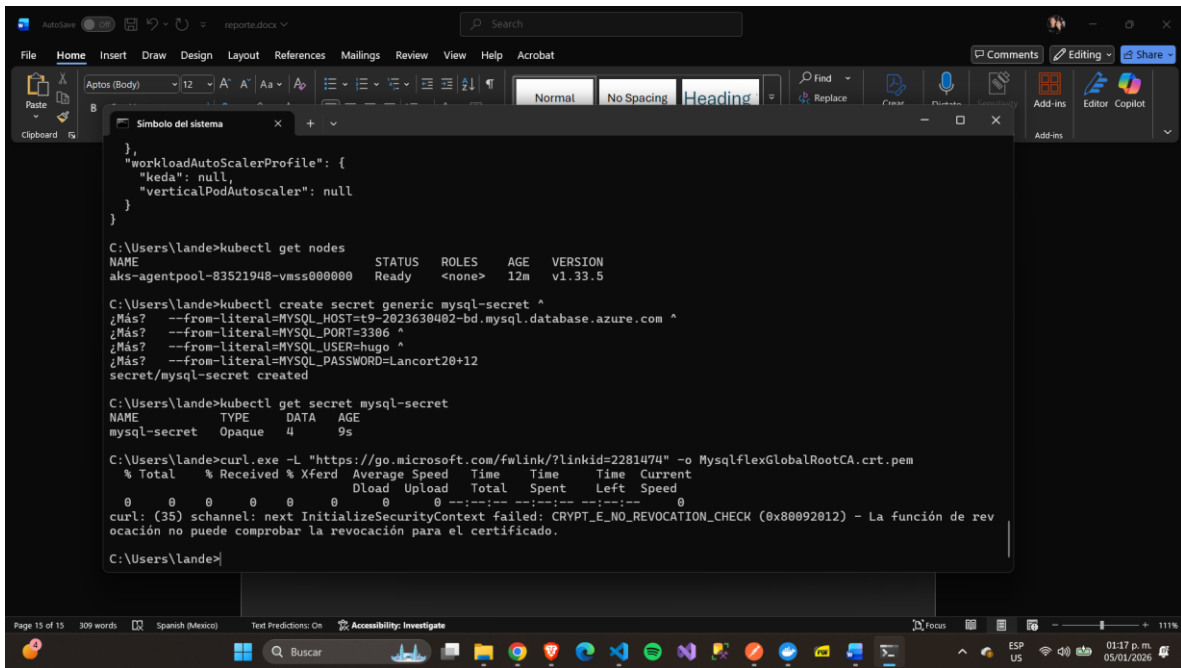


Creacion de tablas y bases de datos

Los scripts utilizados fueron los mismos que de manera local



Creacion mysql-secret



```
},
"workloadAutoScalerProfile": {
  "keda": null,
  "verticalPodAutoscaler": null
}
}

C:\Users\lande>kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-agentpool-83521948-vmss000000 Ready     <none>   12m   v1.33.5

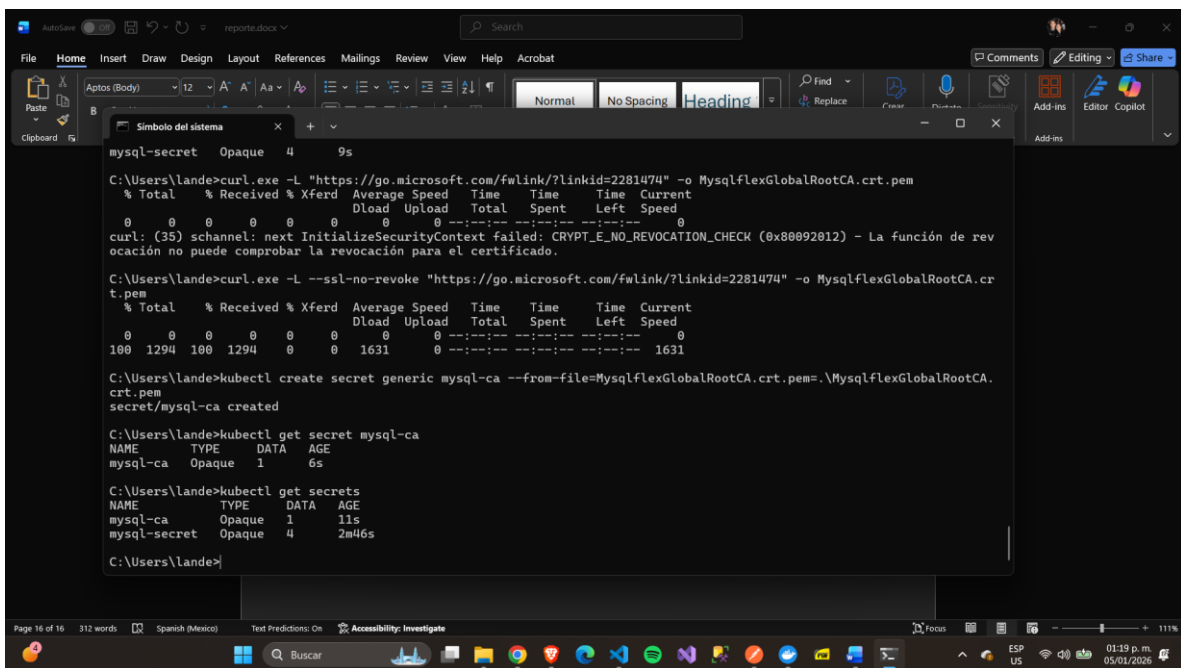
C:\Users\lande>kubectl create secret generic mysql-secret ^
¿Más? --from-literal=MYSQL_HOST=t9-2023630402-bd.mysql.database.azure.com ^
¿Más? --from-literal=MYSQL_PORT=3306 ^
¿Más? --from-literal=MYSQL_USER=hugo ^
¿Más? --from-literal=MYSQL_PASSWORD=Lancort20+12
secret/mysql-secret created

C:\Users\lande>kubectl get secret mysql-secret
NAME      TYPE      DATA   AGE
mysql-secret Opaque    4        9s

C:\Users\lande>curl.exe -L "https://go.microsoft.com/fwlink/?linkid=2281474" -o MysqlflexGlobalRootCA.crt.pem
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (35) schannel: next InitializeSecurityContext failed: CRYPT_E_NO_REVOCATION_CHECK (0x80092012) - La función de rev
ocación no puede comprobar la revocación para el certificado.

C:\Users\lande>
```

Creacion de secrets



```
mysql-secret Opaque 4 9s

C:\Users\lande>curl.exe -L "https://go.microsoft.com/fwlink/?linkid=2281474" -o MysqlflexGlobalRootCA.crt.pem
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
curl: (35) schannel: next InitializeSecurityContext failed: CRYPT_E_NO_REVOCATION_CHECK (0x80092012) - La función de rev
ocación no puede comprobar la revocación para el certificado.

C:\Users\lande>curl.exe -L --ssl-no-revoke "https://go.microsoft.com/fwlink/?linkid=2281474" -o MysqlflexGlobalRootCA.cr
t.pem
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100 1294 100 1294 0 0 1631 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1631

C:\Users\lande>kubectl create secret generic mysql-ca --from-file=MysqlflexGlobalRootCA.crt.pem=.MysqlflexGlobalRootCA.
crt.pem
secret/mysql-ca created

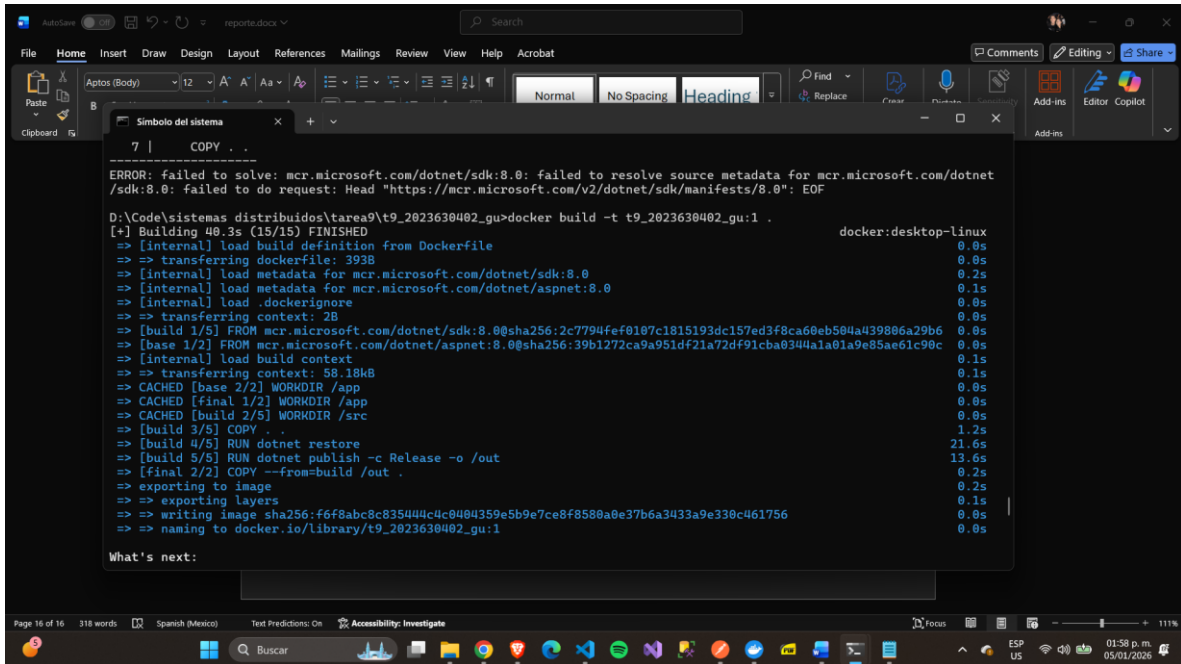
C:\Users\lande>kubectl get secret mysql-ca
NAME      TYPE      DATA   AGE
mysql-ca Opaque    1        6s

C:\Users\lande>kubectl get secrets
NAME      TYPE      DATA   AGE
mysql-ca Opaque    1       11s
mysql-secret Opaque    4      2m46s

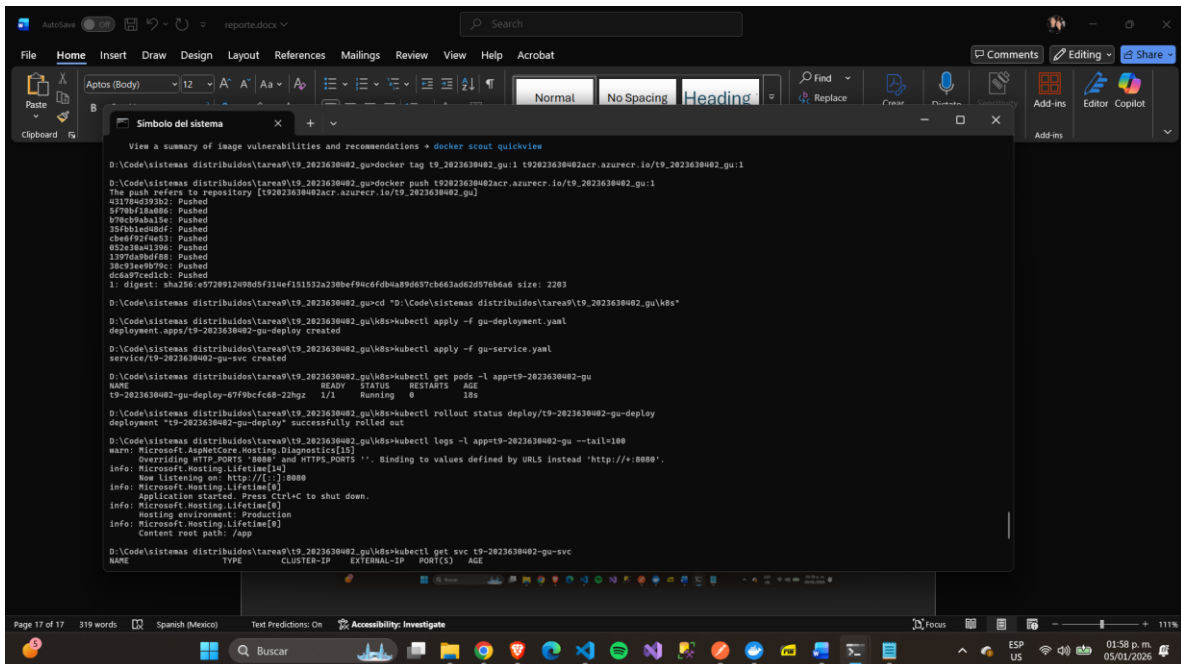
C:\Users\lande>
```

Deployment de contenedores docker al ACR

GU

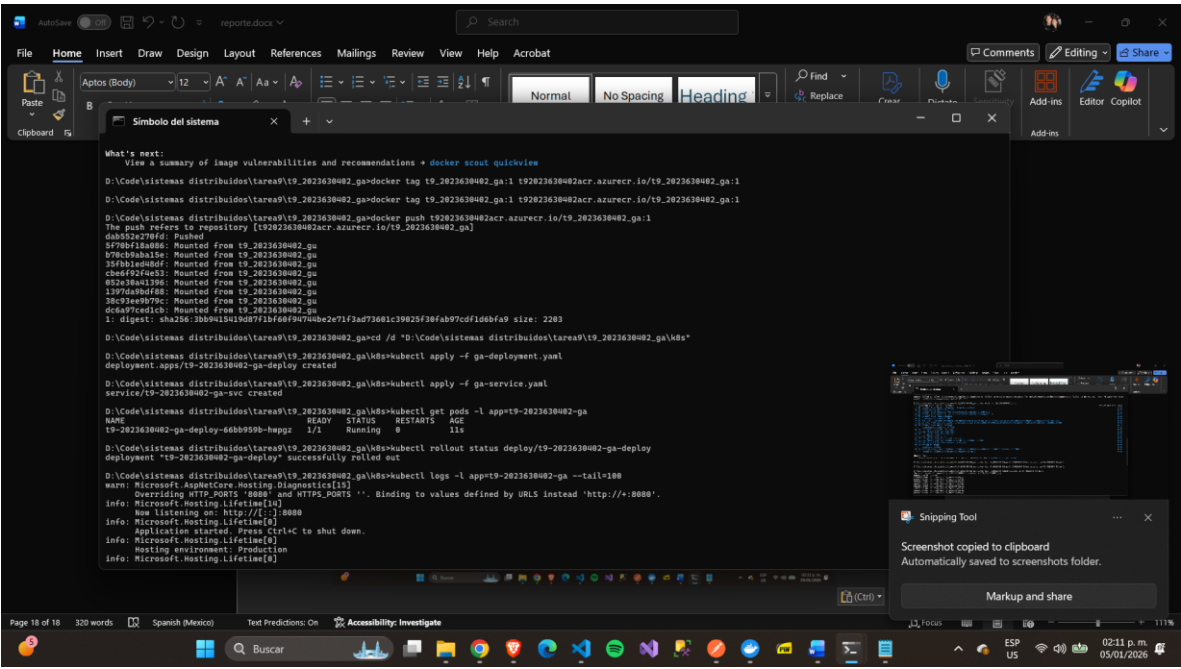
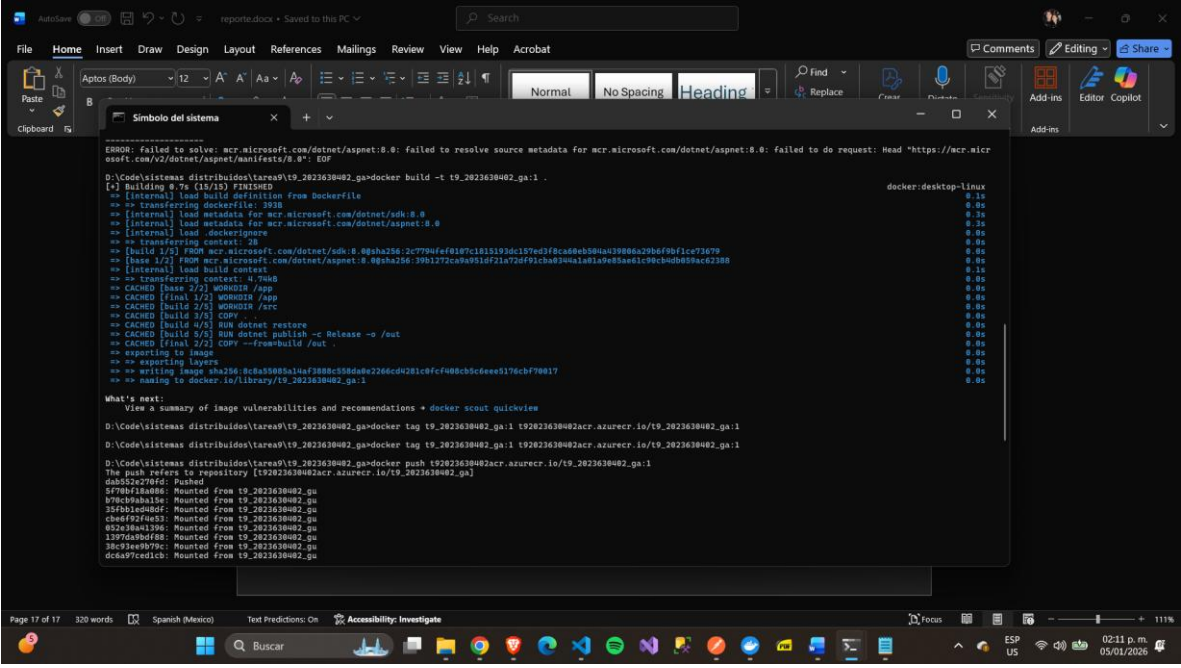


The screenshot shows the Adobe Acrobat interface with a terminal window titled "Símbolo del sistema" open. The terminal displays the output of a Docker build command. It starts with an error message: "ERROR: failed to solve: mcr.microsoft.com/dotnet/sdk:8.0: failed to resolve source metadata for mcr.microsoft.com/dotnet/sdk:8.0: failed to do request: Head \"https://mcr.microsoft.com/v2/dotnet/sdk/manifests/8.0\": EOF". This is followed by the successful execution of the build command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>docker build -t t9_2023630402_gu:1 .". The build progress is shown with various steps and their durations, including "Building 40.3s (15/15) FINISHED", "transferring dockerfile: 3938", "transferring context: 2B", and "exporting image sha256:f6f8abc8c835444c4c0404359e5b9e7ce8f8580a0e37b6a3433a9e330c461756". The final output is "What's next:".

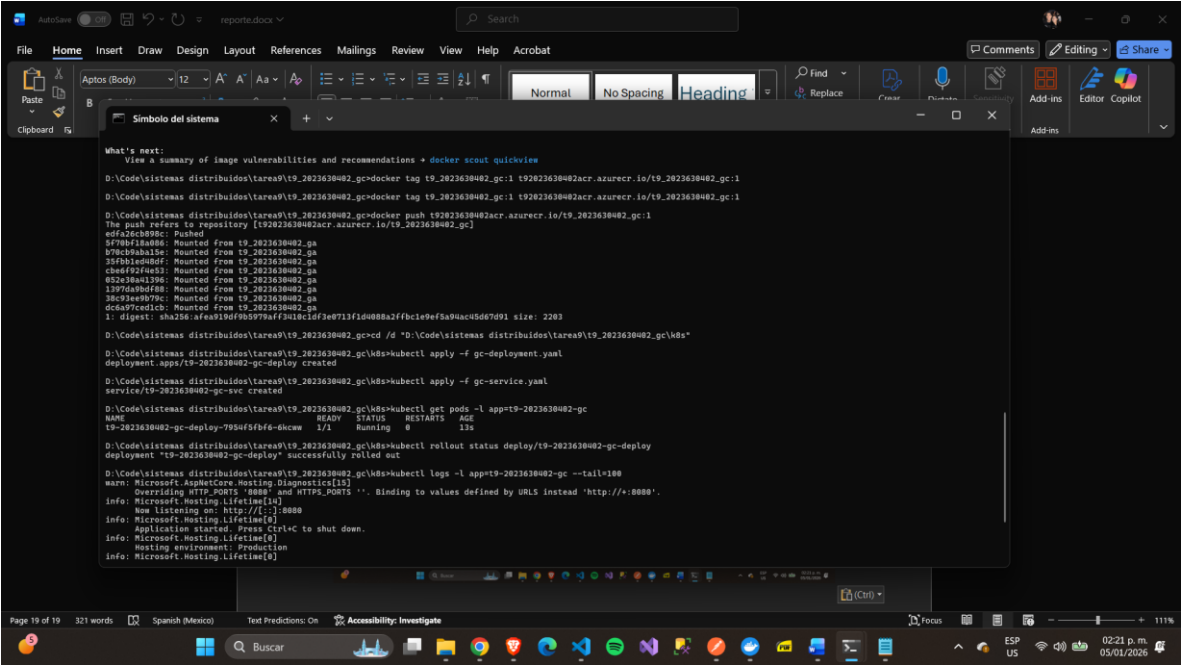
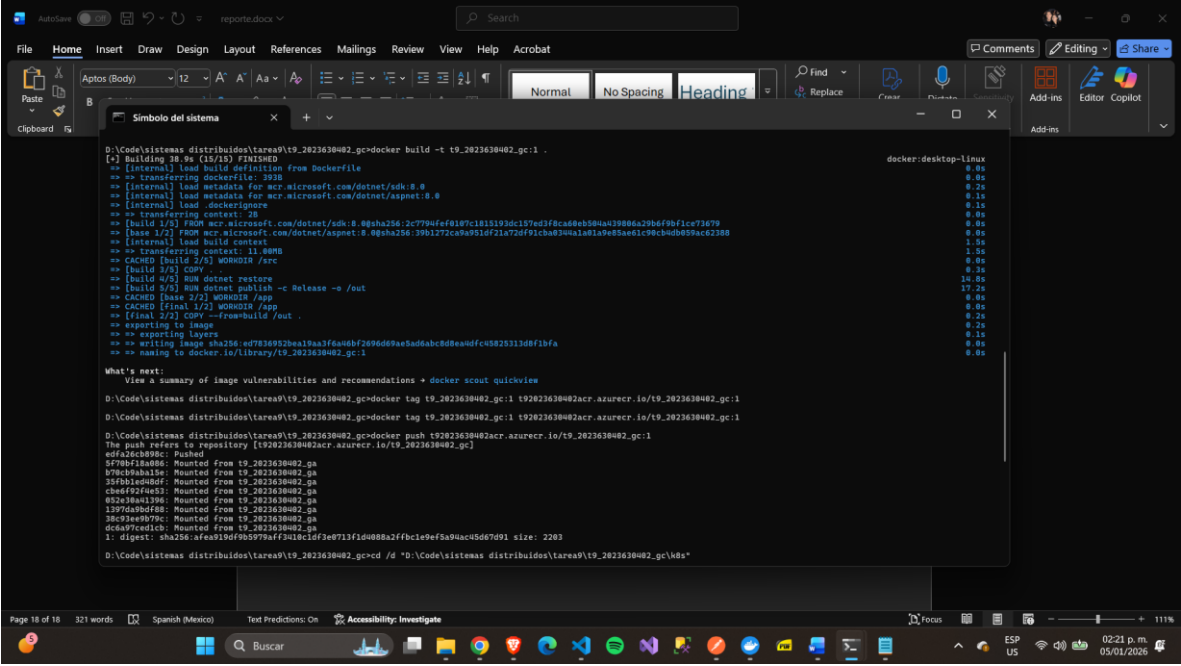


The screenshot shows the Adobe Acrobat interface with a terminal window titled "Símbolo del sistema" open. The terminal displays the output of a Docker push command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>docker tag t9_2023630402_gu:1 t92023630402acr.azurecr.io/t9_2023630402_gu:1". This is followed by the successful execution of the push command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>docker push t92023630402acr.azurecr.io/t9_2023630402_gu:1". The output shows the image being pushed to the repository. The terminal then shows the execution of a deployment command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>kubectl apply -f gu-deployment.yaml". The output shows the deployment being created. The terminal then shows the execution of a rollout command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>kubectl rollout status deploy/t9-2023630402-gu-deploy". The output shows the rollout being successful. The terminal then shows the execution of a logs command: "D:\Code\sistemas distribuidos\tarea9\t9_2023630402_gu>kubectl logs -l app=t9-2023630402-gu --tail=100". The output shows the logs of the application, including the message "Application started. Press Ctrl+C to shut down.".

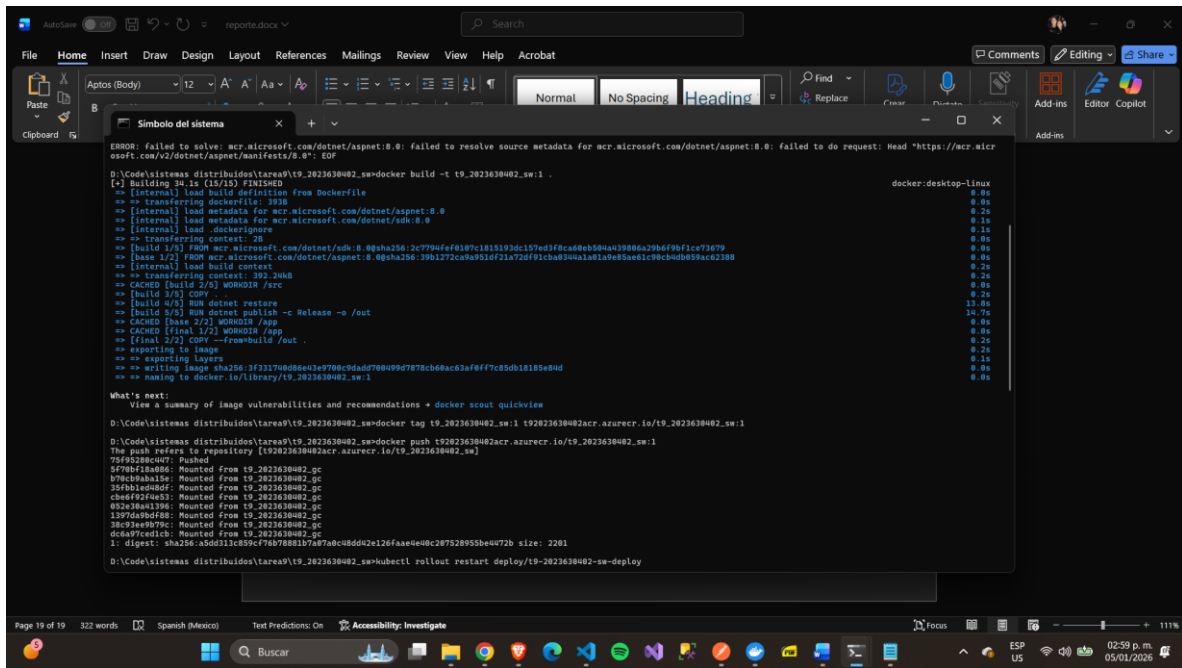
GA



GC



SW



The screenshot shows the Adobe Acrobat interface with a terminal window open. The terminal displays the following commands and output:

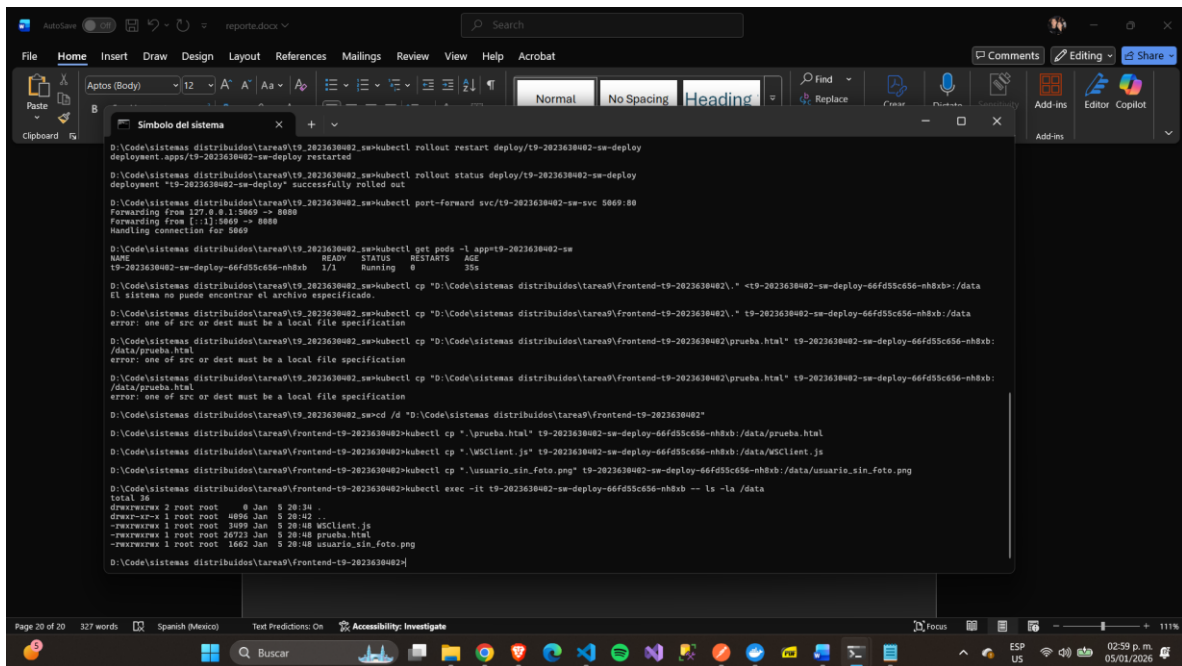
```
D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>docker build -t t9-2023630402_sw:1 .
[+] Building 34.1s (15/15) FINISHED
==> [internal] load build definition from Dockerfile
==> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
==> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
==> [internal] load dockerignore
==> [internal] transfer context: 202.2mb
==> [build 1/5] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:2c779afef8107c1815193dc157ed3f8cad0eb504439886a29d6f9b1ce77679
==> [build 1/5] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:20b1272c8a5016721572d9f1cb8b3d1a5f8a5e1c8eb0b58ac42388
==> [internal] load build context
==> [transfer] context: 202.2mb
==> [cached] build 2/5] WORKDIR /src
==> [build 3/5] COPY . .
==> [build 4/5] RUN dotnet restore
==> [build 5/5] RUN dotnet publish -c Release -o /out
==> [cached] base 3/2] WORKDIR /app
==> [cached] [final 1/2] WORKDIR /app
==> [final 2/2] COPY --from=build /out .
==> exporting to image
==> exporting layers
==> writing image sha256:3f33170d06e43e9780c9dad708099d787b60a6c3af6ff7c85db1818e8ud
==> naming to docker.io/library/t9-2023630402_sw:1

What's next:
View a summary of image vulnerabilities and recommendations + docker scout quickview

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>docker tag t9-2023630402_sw:1 t9-2023630402acr.azurecr.io/t9-2023630402_sw:1
The push refers to repository [t9-2023630402acr.azurecr.io/t9-2023630402_sw]
75f95280c407: Pushed
5798b1f1a86: Mounted from t9-2023630402_sw
870c90a15a: Mounted from t9-2023630402_sw
35fb51e4d4f: Mounted from t9-2023630402_sw
c4ed492e4e3: Mounted from t9-2023630402_sw
082a30a1396: Mounted from t9-2023630402_sw
1397d4bdf6e: Mounted from t9-2023630402_sw
38c39e9979c: Mounted from t9-2023630402_sw
dc8af7cd1cb: Mounted from t9-2023630402_sw
1: digest: sha256:ab0d11c89c70d708137a7a80c8dd42e126fae4e0c297528955b4e472b size: 2201

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl rollout restart deploy/t9-2023630402-sw-deploy
```

Subida de archivos del front



The screenshot shows the Adobe Acrobat interface with a terminal window open. The terminal displays the following commands and output:

```
D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl rollout restart deploy/t9-2023630402-sw-deploy
deployment.apps/t9-2023630402-sw-deploy restarted

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl rollout status deploy/t9-2023630402-sw-deploy
deployment "t9-2023630402-sw-deploy" successfully rolled out

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl port-forward svc/t9-2023630402-sw-svc 5069:80
Forwarding from 127.0.0.1:5069 -> 8080
Handling connection for 5069

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl get pods -l app=t9-2023630402-sw
NAME READY STATUS RESTARTS AGE
t9-2023630402-sw-deploy-66fd55c656-nhxb 1/1 Running 0 35s

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl cp "D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402\" t9-2023630402-sw-deploy-66fd55c656-nhxb:/data
E! sistema no puede encontrar el archivo especificado

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl cp "D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402\" t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/prueba.html
error: one of src or dest must be a local file specification

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl cp "D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402\prueba.html" t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/prueba.html
error: one of src or dest must be a local file specification

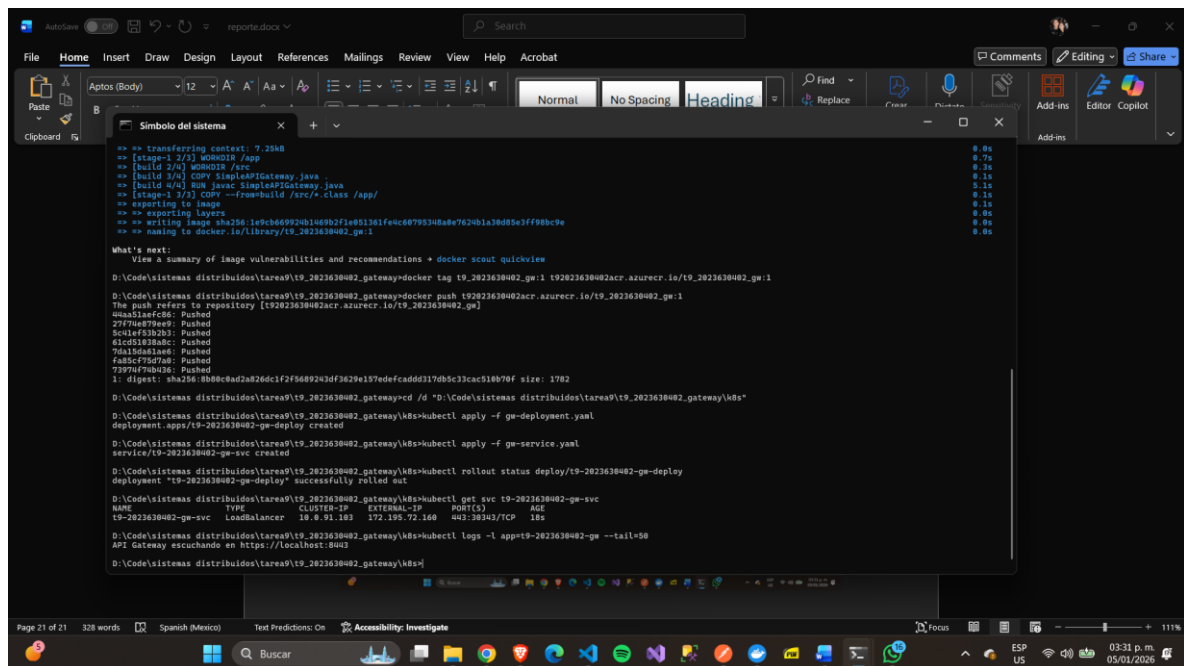
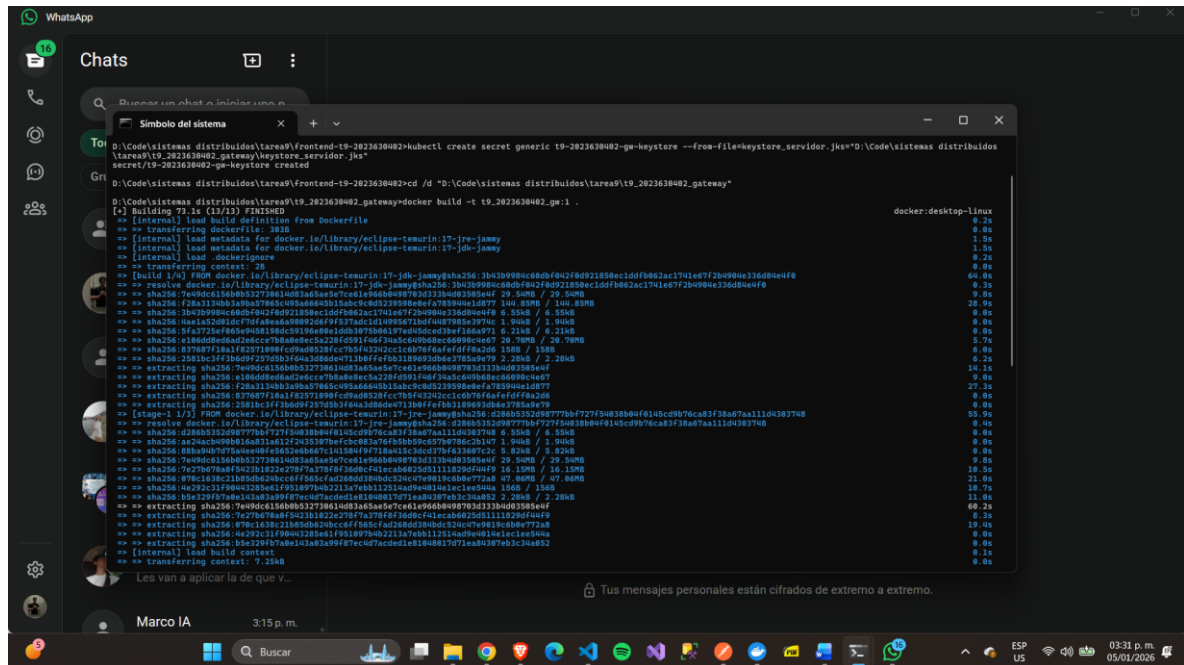
D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>kubectl cp "D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402\prueba.html" t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/prueba.html
error: one of src or dest must be a local file specification

D:\Code\sistemas distribuidos\tarea9\t9-2023630402_sw>cd /d "D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402"
D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402>kubectl cp *.prueba.html t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/prueba.html
D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402>kubectl cp *.WSClient.js t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/WSClient.js
D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402>kubectl cp *.usuario_sin_foto.png t9-2023630402-sw-deploy-66fd55c656-nhxb:/data/usuario_sin_foto.png
D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402>kubectl exec -it t9-2023630402-sw-deploy-66fd55c656-nhxb -- ls -la /data
total 36
drwxrwxrwx 2 root root 4 Jan 5 20:30
drwxrwxrwx 1 root root 4096 Jan 5 20:42 .
-rwxrwxrwx 1 root root 3499 Jan 5 20:48 WSClient.js
-rwxrwxrwx 1 root root 2072 Jan 5 20:48 prueba.html
-rwxrwxrwx 1 root root 1662 Jan 5 20:48 usuario_sin_foto.png

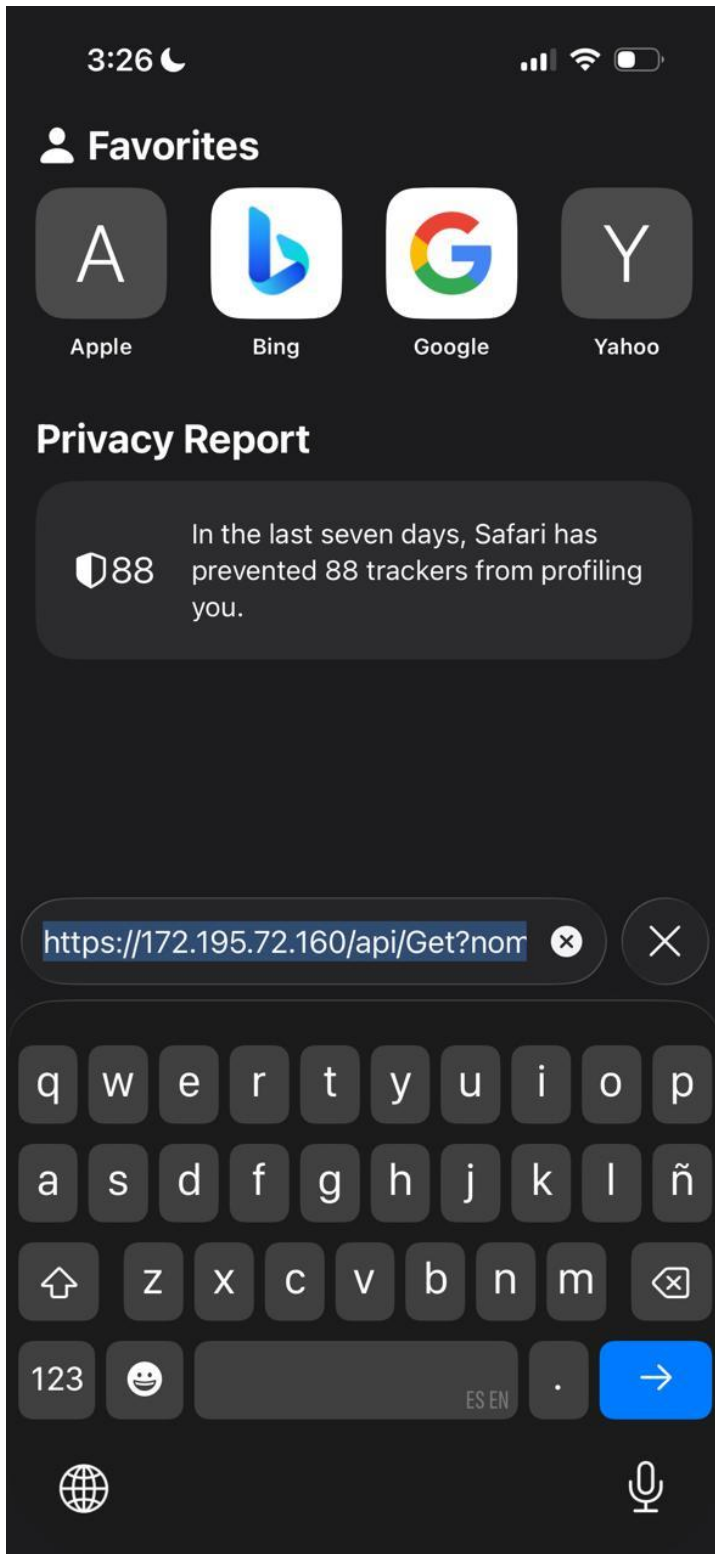
D:\Code\sistemas distribuidos\tarea9\frontend-t9-2023630402>
```

Gateway

Subida de certificado auto firmado y creación de balanceador de carga que expone una ip pública.



Pruebas front desde ip publica



3:26



Acceso al sistema

Email

Password

Aceptar

Registro de usuario



172.195.72.160



3:28



Registro de usuario

Email *

jonas@gmail.com

Contraseña *

••••

Nombre *

Jonas

Apellido paterno *

Landeros

Apellido materno

Cortes

Fecha de nacimiento *

4 Jan 2026 at 3:27 p.m.

Teléfono

5546876467

Genero



Choose Files



IMG_6129.jpeg

Registrar usuario

Cancelar

3:28



Registro de usuario

Email *

jonas@gmail.com

Contraseña *

••••

Nombre *

Jonas

Apellido paterno *

Landeros

Apellido materno

Cortes

Fecha de nacimiento *

Se registró el usuario

Close



Choose Files



IMG_6129.jpeg

Registrar usuario

3:28



Acceso al sistema

Email

jonas@gmail.com

Password

•••4

Aceptar

Registro de usuario

172.195.72.160



3:28



Perfil del usuario

Captura de artículo

Compra de artículos

Eliminar perfil

Salir



172.195.72.160



3:28



Perfil del usuario

Email *

jonas@gmail.com

Contraseña *

Nombre *

Jonas

Apellido paterno *

Landeros

Apellido materno

Cortes

Fecha de nacimiento *

4 Jan 2026 at 3:27 p.m.

Teléfono

5546876467

Genero



Choose Files no files selected

Quitar foto

Guardar cambios

Regresar



172.195.72.160



3:29



Captura de artículo

Nombre *

Prueba 1

Descripción *

Jsjh

Precio *

12

Existencia *

123



Choose Files



IMG_6437.jpeg

Guardar artículo

Regresar



172.195.72.160



3:29



Captura de artículo

Nombre *

Prueba 1

Descripción *

Jsh

Precio *

12

Existencia *

123

Se dio de alta el artículo

Close

Choose Files IMG_6437.jpeg

Guardar artículo

Regresar



172.195.72.160



3:29



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Prueba 1

Jsjh

Precio: \$12

Cantidad:

Compra

Regresar al menú



172.195.72.160



3:30

Carrito de compra



Prueba 1

Jsjh

Precio: \$12

Cantidad:

Compra

Regresar al menú

172.195.72.160



"3"

1

2

3

4

5

6

7

8

9

0

-

/

:

;

(

)

\$

&

@

"

#+=

.

,

?

!

'



ABC

ES EN



3:30



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Se agregó el artículo al carrito

Close

Cantidad:

Compra

Regresar al menú



172.195.72.160



3:30



Artículos en el carrito



Prueba 1

Precio: \$12

Cantidad: 3

Subtotal: \$36.00

Eliminar artículo

Total: \$36.00

Eliminar carrito

Seguir comprando

Regresar al menú



172.195.72.160



3:30



Artículos en el carrito



Prueba 1

Precio: \$12

Cantidad: 3

Subtotal: \$36.00

Eliminar artículo

¿Deseas eliminar todos los artículos del carrito?

Cancel OK

Regresar al menú



172.195.72.160



3:30



Artículos en el carrito



Prueba 1

Precio: \$12

Cantidad: 3

Subtotal: \$36.00

Eliminar artículo

Total: \$36.00

Se eliminó el carrito de compra

Close

Seguir comprando

Regresar al menú



172.195.72.160



3:30



Artículos en el carrito

El carrito está vacío

Total: \$0.00

Eliminar carrito

Seguir comprando

Regresar al menú



172.195.72.160



3:30



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Prueba 1

Jsjh

Precio: \$12

Cantidad:

Compra

Regresar al menú



172.195.72.160



3:30



Perfil del usuario

Captura de artículo

Compra de artículos

Eliminar perfil

Salir

¿Deseas eliminar el perfil del usuario?

Cancel OK



172.195.72.160



3:30



Perfil del usuario

Captura de artículo

Compra de artículos

Eliminar perfil

Salir

Se eliminó el perfil del usuario

Close

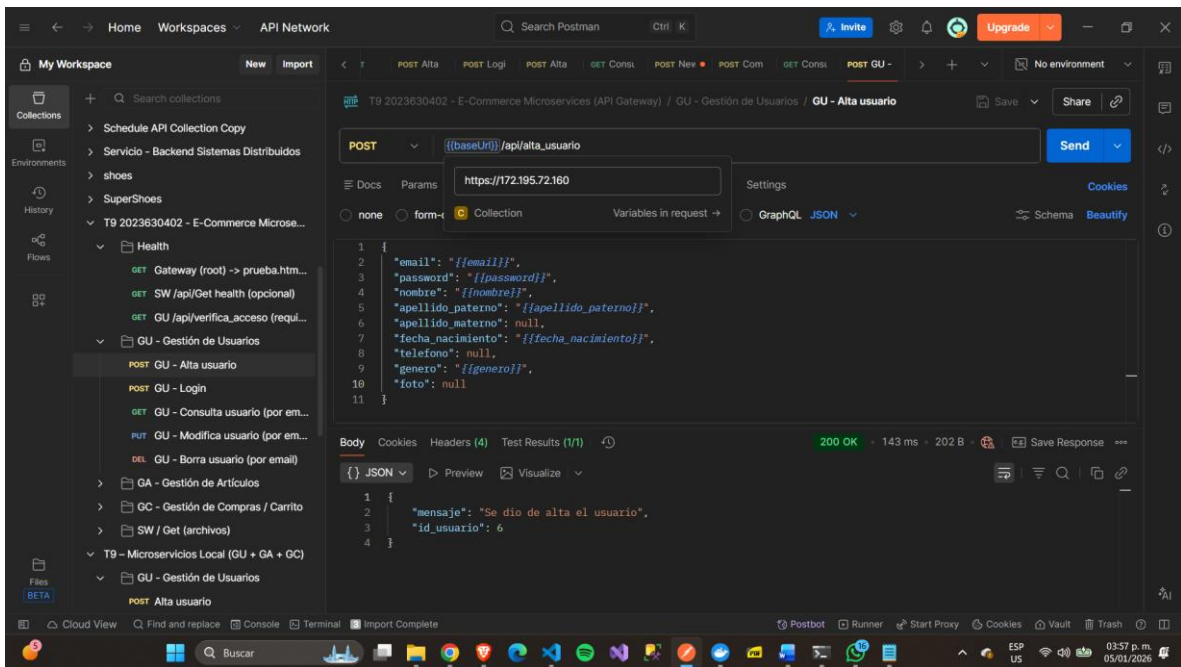


172.195.72.160

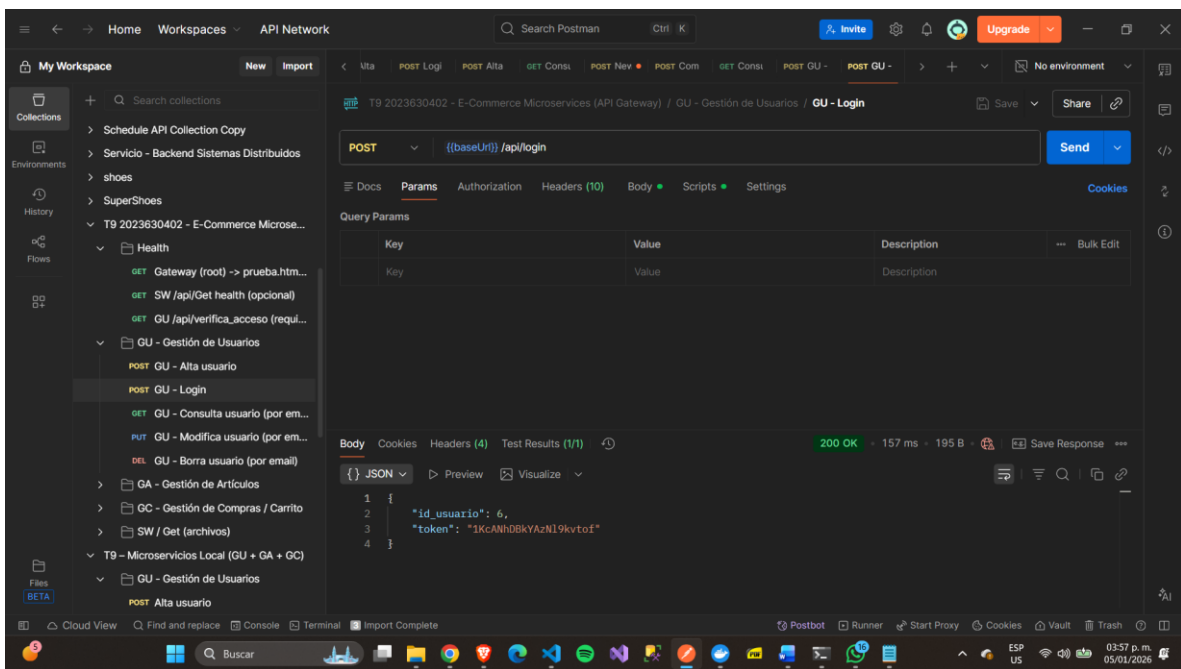


Pruebas backend

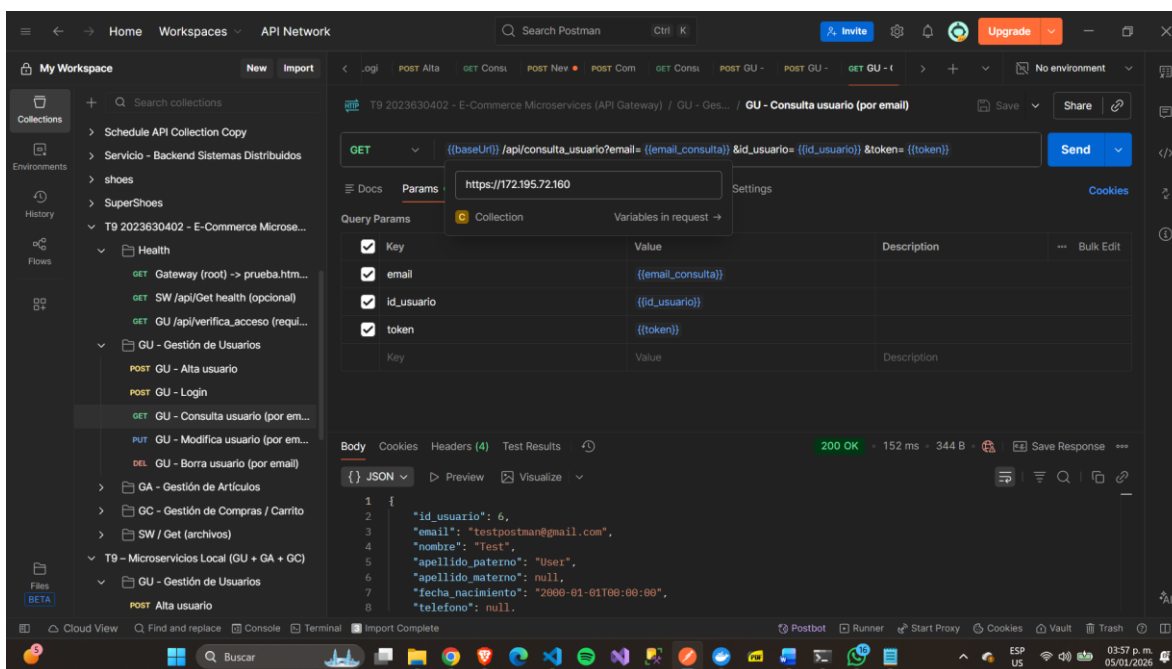
Registrar usuario



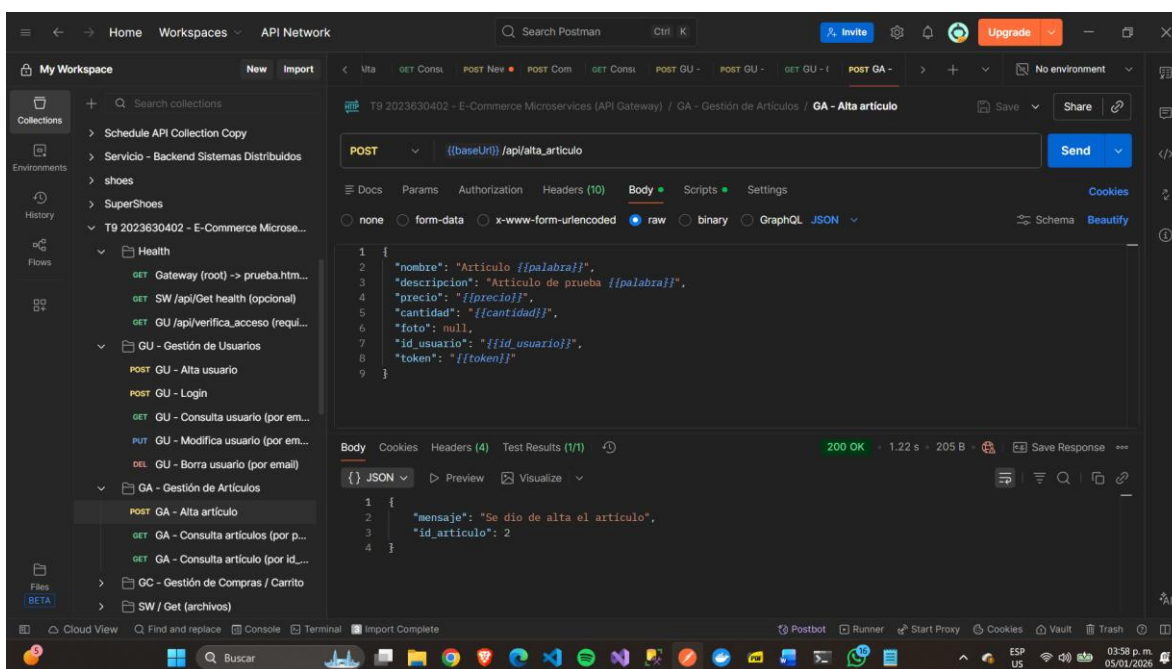
Login



Obtener usuario



Crear articulo



Consultar artículos

The screenshot shows the Postman interface with a GET request configured. The URL is `{{baseUrl}}/api/consulta_articulos?palabra={{palabra}} &id_usuario={{id_usuario}} &token={{token}}`. The base URL is set to `https://172.195.72.160`. The request is sent, and the response is a 200 OK status with a JSON body:

```
{  "id_articulo": 2,  "nombre": "Articulo test",  "descripcion": "Articulo de prueba test",  "precio": 99.99,  "foto": null}
```

The left sidebar shows a collection of API endpoints under the workspace "T9 2023630402 - E-Commerce Microservices (API Gateway)".

Comprar artículo

The screenshot shows the Postman interface with a POST request configured. The URL is `{{baseUrl}}/api/compra_articulo`. The base URL is set to `https://172.195.72.160`. The request is sent, and the response is a 200 OK status with a JSON body:

```
{  "mensaje": "Articulo agregado al carrito"}
```

The left sidebar shows a collection of API endpoints under the workspace "T9 2023630402 - E-Commerce Microservices (API Gateway)".

Consulta carrito

The screenshot shows the Postman interface with a workspace named 'API Network'. The left sidebar displays a collection of API endpoints under 'GC - Gestión de Compras / Carrito'. The selected endpoint is 'GC - Consulta carrito' (GET). The URL is set to 'https://172.195.72.160/api/consulta_carrito_compra?id_usuario={{id_usuario}}&token={{token}}'. The 'Params' tab is active, showing query parameters 'id_usuario' and 'token'. The 'Send' button is highlighted. The response is a 200 OK status with a 159 ms response time and 272 B of data. The response body is displayed in JSON format:

```
{
  "id_articulo": 2,
  "nombre": "Articulo test",
  "descripcion": "Articulo de prueba test",
  "precio": 99.99,
  "cantidad": 2,
  "foto": null
}
```

Modificar carrito

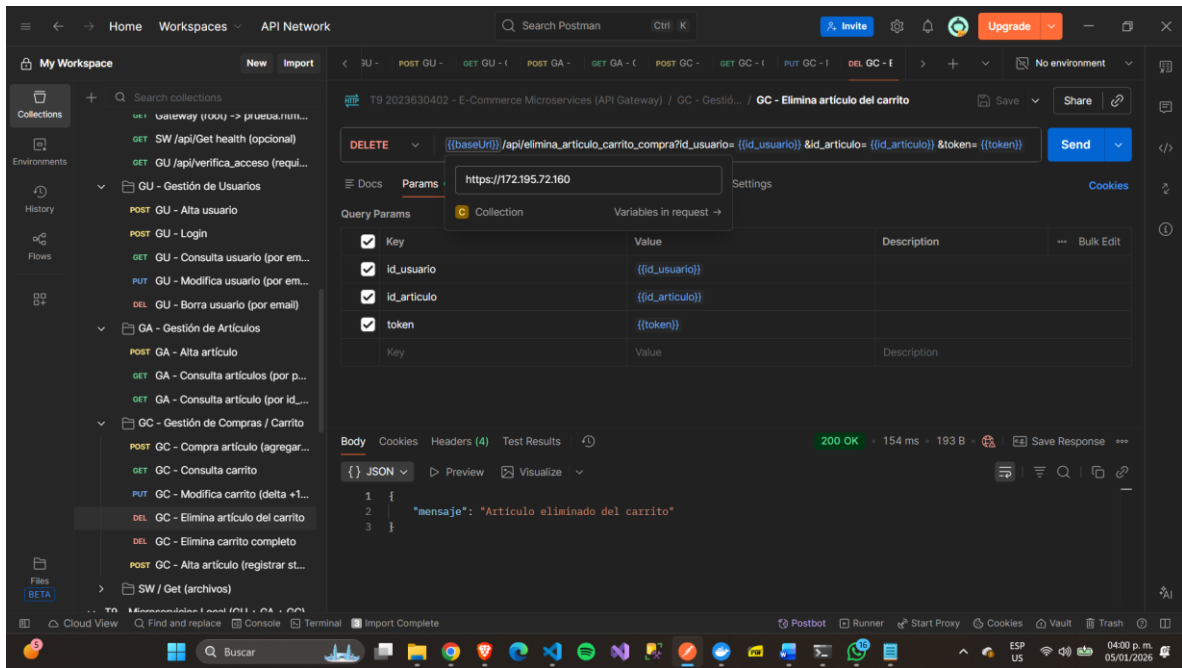
The screenshot shows the Postman interface with a workspace named 'API Network'. The left sidebar displays a collection of API endpoints under 'GC - Gestión de Compras / Carrito'. The selected endpoint is 'GC - Modifica carrito (delta +1/-1)' (PUT). The URL is set to 'https://172.195.72.160/api/modifica_carrito_compra'. The 'Body' tab is active, showing a raw request body with the following JSON:

```
{
  "id_usuario": "{{id_usuario}}",
  "token": "{{token}}",
  "id_articulo": "{{id_articulo}}",
  "delta": "{{delta}}"
}
```

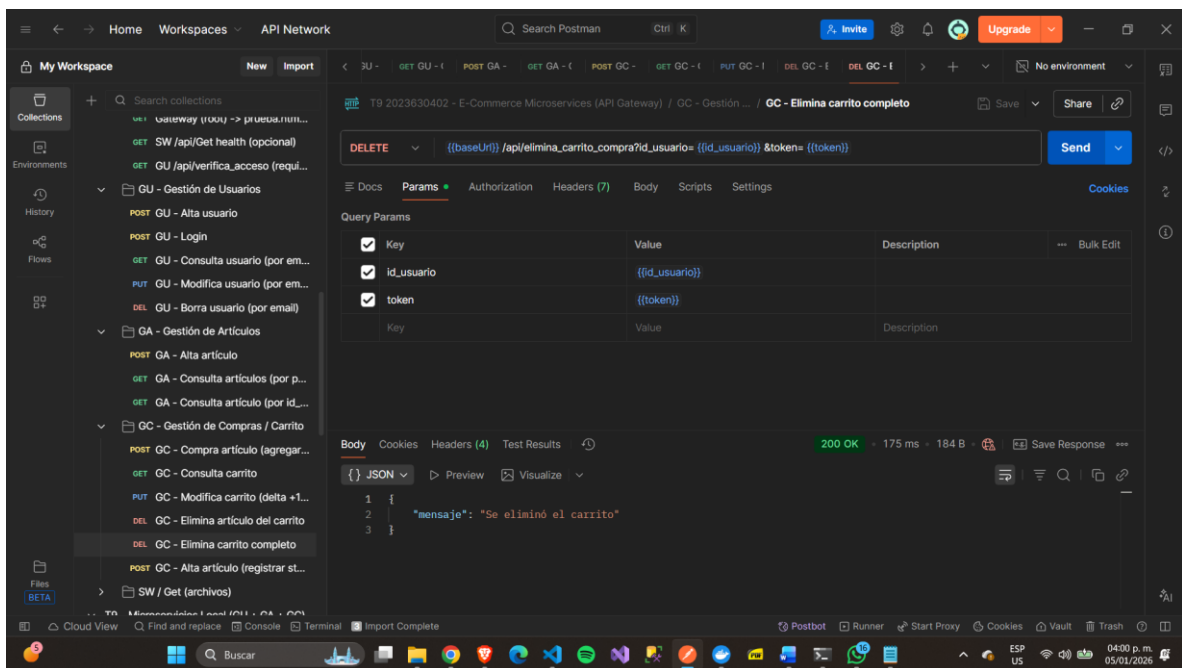
The 'Send' button is highlighted. The response is a 400 Bad Request status with a 180 ms response time and 209 B of data. The response body is displayed in JSON format:

```
{
  "mensaje": "No hay suficientes articulos en stock"
}
```

Eliminar artículo del carrito



Eliminar carrito



Explicacion de microservicios

2.1 GU – Gestión de Usuarios (t9_2023630402_gu)

Responsabilidad: administrar usuarios, autenticación y verificación de acceso.

Base de datos: MySQL (PaaS Azure) con tablas tipo:

- usuarios (datos del usuario, password, token)
- fotos_usuarios (foto opcional en BLOB)

Endpoints principales:

- POST /gu/alta_usuario
Registra un usuario. Recibe datos personales y opcionalmente una foto en base64. Devuelve id_usuario.
- POST /gu/login
Valida credenciales (email + password) y genera un token que se guarda en DB. Devuelve { id_usuario, token }.
- GET /gu/verifica_acceso?id_usuario=...&token=...
Verifica que el token corresponda al usuario (control de acceso). Devuelve true/false.
- GET /gu/consulta_usuario?email=...&id_usuario=...&token=...
Consulta datos de un usuario (requiere token válido).
- PUT /gu/modifica_usuario?email=...&id_usuario=...&token=...
Modifica datos del usuario (y opcionalmente password / foto).
- DELETE /gu/borra_usuario?email=...&id_usuario=...&token=...
Elimina usuario y foto asociada.

Función clave del sistema:

GU es el “servicio de identidad”: los demás microservicios **no autentican por sí mismos**, sino que **validan acceso llamando a GU** (/gu/verifica_acceso).

2.2 GA – Gestión de Artículos (t9_2023630402_ga)

Responsabilidad: alta y consulta de artículos (catálogo).

Base de datos: MySQL con tablas tipo:

- stock (artículos: id, nombre, descripción, precio)
- fotos_articulos (foto opcional en BLOB)

Dependencias internas:

- Llama a **GU** para verificar token.

- Llama a **GC** para registrar stock inicial al crear un artículo.

Endpoints principales:

- POST /ga/alta_articulo
Da de alta un artículo (nombre, descripción, precio, foto opcional, cantidad, id_usuario, token).
Flujo:
 1. Verifica acceso en GU
 2. Inserta artículo en GA (y foto si existe)
 3. Llama a GC (/gc/alta_articulo) para registrar cantidad inicial
 4. Devuelve id_articulo
- GET /ga/consulta_articulos?palabra=...&id_usuario=...&token=...
Busca artículos por coincidencia en nombre o descripción (requiere token).
- GET /ga/consulta_articulo?id_articulo=...&id_usuario=...&token=... (**nuevo**)
Devuelve un artículo específico por ID. Este endpoint se usa en GC para enriquecer el carrito con datos del artículo.

2.3 GC – Gestión de Compras / Carrito (t9_2023630402_gc)

Responsabilidad: stock transaccional y operaciones de carrito/compra.

Base de datos: MySQL con tablas tipo:

- stock (existencias por id_articulo)
- carrito_compra (id_usuario, id_articulo, cantidad)

Dependencias internas:

- Llama a **GU** para verificar token.
- Llama a **GA** para obtener detalles del artículo cuando se consulta el carrito (nombre, foto, precio, etc.).

Endpoints principales:

- POST /gc/alta_articulo
Registro de stock inicial por id_articulo (llamado desde GA).

- POST /gc/compra_articulo
Agrega al carrito y descuenta stock **en transacción**.
Flujo (con FOR UPDATE):
 1. Bloquea fila de stock
 2. Verifica existencia suficiente
 3. Inserta/actualiza carrito
 4. Descuenta stock
- GET /gc/consulta_carrito_compra?id_usuario=...&token=...
Devuelve contenido del carrito.
Importante: además de devolver id_articulo y cantidad, **consulta GA** (/ga/consulta_articulo) para llenar nombre, precio, foto, etc.
- DELETE
/gc/elimina_articulo_carrito_compra?id_usuario=...&id_articulo=...&token=...
Elimina un artículo del carrito y regresa su cantidad al stock (transacción).
- DELETE /gc/elimina_carrito_compra?id_usuario=...&token=...
Vacía el carrito completo y regresa stock (transacción).
- PUT /gc/modifica_carrito_compra
Modifica cantidad del carrito con un delta (+1 o -1), cuidando stock (transacción).

Microservicio de archivos estáticos (GET / SW) (t9_2023630402_sw)

Responsabilidad: servir archivos del frontend (HTML, JS, imágenes).

Este microservicio implementa un endpoint tipo “/Get” que permite entregar archivos desde una carpeta montada en un volumen persistente.

Endpoints:

- /api/Get?nombre=/archivo (principal)
- /Get?nombre=/archivo (alias)
- / (alias)

Características:

- Usa variable de entorno ROOT=/data/

- Usa un PVC con azurefile para almacenar archivos accesibles por el pod.
- Implementa:
 - Validación básica contra path traversal (..)
 - Last-Modified y If-Modified-Since para caché (responde 304 si aplica)

4) API Gateway (Java) – Punto de entrada único

Responsabilidad: exponer HTTPS y enrutar por ruta hacia microservicios internos.

Ventajas:

- Un solo endpoint público para el cliente (navegador o móvil)
- Simplifica CORS / llamadas del front
- Centraliza el acceso y el ruteo

Funcionamiento:

- Escucha por HTTPS (certificado en keystore_servidor.jks)
- Recibe requests y, según el path, reenvía al microservicio correcto:
 - /api/* → GU / GA / GC
 - /api/Get o / → servicio SW
- Reescribe rutas (por ejemplo /api/login → /gu/login).

Requerimientos funcionales del Front-End

1) Pantalla “Captura de artículo”

- El formulario valida que **nombre, descripción, precio y cantidad** estén presentes y sean válidos.
- Si se incluye foto, el front la convierte a **Base64** (por ejemplo con FileReader) para enviarla como texto en JSON.
- Al enviar, el front debe llamar al endpoint del Gateway:
 - POST /api/alta_articulo
- En el body se manda:
 - nombre, descripcion, precio, cantidad, foto (opcional), id_usuario, token

- Esto cumple el flujo del back-end (GA valida con GU y registra stock inicial en GC).

2) Pantalla “Compra de artículos” con búsqueda

- Se muestra un campo tipo input para “palabra”.
- Al buscar (botón o enter), el front llama:
 - GET /api/consulta_articulos?palabra=...&id_usuario=...&token=...
- La palabra se usa para buscar coincidencias en **nombre** y **descripción** de los artículos (esto lo hace el back en GA).

3) Lista de resultados con controles de cantidad (+ / -) y botón “Compra”

- La foto llega como **base64** (o null). Si viene, se muestra como:
 - src="data:image/png;base64,..." (o jpg dependiendo)
- El front mantiene un valor de cantidad por artículo:
 - mínimo recomendado: **1** (no permitir 0 o negativo)
- Botón “+” incrementa el valor.
- Botón “-” decrementa (hasta 1).

Relación con back-end:

- El “+/-” es control visual del front; la operación real de compra se manda al back.

4) Botón “Compra” realiza la compra (llamada al backend)

- Toma la cantidad seleccionada.
- Hace una petición al Gateway:
 - POST /api/compra_articulo
- En el body manda:
 - id_usuario, token, id_articulo, cantidad
- El back (GC) hace la transacción:
 - valida token con GU
 - verifica stock

- descuento stock
- agrega/actualiza carrito

5) Botón “Carrito de compra” + pantalla “Artículos en el carrito”

Cómo se implementa en el front:

- Al entrar al carrito, el front llama:
 - GET /api/consulta_carrito_compra?id_usuario=...&token=...
- GC devuelve una lista de items, pero ya enriquecida porque GC consulta GA para nombre/foto/precio.
- El front calcula:
 - $\text{costoItem} = \text{cantidad} * \text{precio}$
 - $\text{total} = \text{suma}(\text{costoItem})$
- Renderiza tabla/lista con esos valores.

6) Botón “Eliminar artículo” en cada artículo del carrito

- El botón llama al Gateway:
 - DELETE
/api/elimina_articulo_carrito_compra?id_usuario=...&id_articulo=...&token=...
- Luego de eliminar, el front:
 - refresca el carrito llamando otra vez consulta_carrito_compra
 - recalcula el total

7) Botón “Eliminar carrito”

- Llama al Gateway:
 - DELETE /api/elimina_carrito_compra?id_usuario=...&token=...
- Después:
 - refresca carrito (debería quedar vacío)
 - $\text{total} = 0$

8) Botón “Seguir comprando”

- Es navegación/estado del frontend:
 - Ocultar “Carrito” y volver a “Compra”
- No requiere llamada al backend; solo UI.

Conclusión

Esta práctica me permitió aplicar de forma práctica los conceptos fundamentales de **arquitectura de microservicios** y **sistemas distribuidos**, desarrollando un sistema de comercio electrónico completo y funcional desplegado en la nube. A lo largo del desarrollo comprendí la importancia de dividir un sistema complejo en microservicios independientes, cada uno con una responsabilidad específica, lo que facilita el mantenimiento, la escalabilidad y la evolución del sistema.

Aprendí a diseñar y consumir **APIs REST**, así como a manejar la comunicación entre microservicios de manera segura mediante **tokens de autenticación**, lo cual fue fundamental para garantizar el acceso controlado a las funcionalidades del sistema. También entendí cómo coordinar operaciones distribuidas, como el proceso de compra, en donde intervienen múltiples microservicios y bases de datos, asegurando la consistencia de la información.

Durante la práctica reforcé el uso de **Docker** para la contenerización de aplicaciones y **Kubernetes** para la orquestación de microservicios, logrando desplegar, administrar y monitorear los servicios en un entorno real de Azure. La implementación del **API Gateway en Java** me permitió centralizar el acceso al sistema, definir un único punto de entrada seguro mediante HTTPS y facilitar la comunicación entre el front-end y los distintos microservicios, incluso desde dispositivos móviles.

Además, comprendí la importancia del uso de **volúmenes persistentes** para el manejo de archivos estáticos del front-end y el uso de **secretos y variables de entorno** para proteger información sensible como credenciales de bases de datos. Finalmente, fortalecí mis habilidades de prueba y depuración utilizando herramientas como **Postman**, validando el correcto funcionamiento de todo el flujo del sistema.

En conclusión, esta práctica consolidó mis conocimientos teóricos y prácticos sobre sistemas distribuidos y microservicios, brindándome una experiencia cercana a un entorno profesional real y preparándome para el desarrollo de aplicaciones modernas escalables y seguras.

Link al repositorio:

<https://github.com/PedroLanderos/SistemasDistribuidos/tree/main/tarea9>